Rewind : Microarchitecture Reverse Engineering Expeiments Across Different Processors

Justin Kuethe

November 2, 2026

GitHub repository: `https://github.com/NovaCorz/CSC591-Project3`
Overleaf project: `https://www.overleaf.com/project/6ad79809eb5b3ac52decd2f7`

Abstract

This paper presents an evaluation of processor microarchitecture across eight different machines, examining architectural features through a series of performance tests. The experiments investigate instruction pipelining, out-of-order execution, branch prediction, reorder buffer capacity, TLB behavior, branch target buffer capacity, and pipeline depth. Measurements are collected and analyzed to identify architectural characteristics, compare performance across processors, and examine differences between performance and efficiency cores in SGX active processors and hybrid CPU designs. The results demonstrate that processors employ different execution and memory-management mechanisms, while also highlighting differences in their measured capacities, latencies, and branch recovery behavior. Overall, this study illustrates how designed experiments can reveal underlying processor architecture through observable performance characteristics.

1 Introduction

Modern processors incorporate microarchitectural features designed to improve instruction throughput, reducee latency, and efficiently manage memory operations. Although processor manufacturers document some architectural specifications, many important details, such as reorder buffer capacity, branch prediction behavior, and translation lookaside buffer (TLB) performance, are not always publicly available or directly observable. Understanding these characteristics provides insight into how processors execute instructions and respond to different workloads.

This project investigates the microarchitectural characteristics of eight machines using a collection of targeted experiments. The experiments examine instruction pipelining, out-of-order execution, branch prediction, reorder buffer capacity, TLB miss latency, branch target buffer (BTB) capacity, and effective pipeline depth. By analyzing execution times, latency measurements, and performance changes under controlled workloads, the underlying behavior of each processor can be inferred.

A particular focus is placed on comparing architectural differences across processor generations and designs, including SGX active processors, as well as hybrid architectures that contain separate performance (P) and efficiency (E) cores. These comparisons help illustrate how different core designs balance performance, resource utilization, and execution efficiency. Ultimately, this project demonstrates how software-based experimentation can be used to investigate processor architecture and identify meaningful differences between modern CPU implementations.

2 Machine Overview

Eight machines were tested on throughout the exercises. All of the machines are x86_64 architecture. Five machines, Sunbird, Charnwood, Skylark, Ookay, and Crux did not have SGX activated or are hybrid processors. One other machine, Artemisia, have SGX activated and are not hybrid processors. The remaining two machines, Elevation and Descent, are hybrid processors with no SGX active.

Machine	CPU	Vendor	ISA	Microarch.	Year
Sunbird	2× Xeon E5-2680 v3	Intel	x86-64	Haswell	2014
Charnwood	Core i7-6700	Intel	x86-64	Skylake	2015
Ookay	Core i7-7700	Intel	x86-64	Kaby Lake	2017
Crux	Core i7-9700	Intel	x86-64	Coffee Lake Refresh	2018
Skylark	2× AMD EPYC 7532	AMD	x86-64	Zen 2 / Rome	2019
Elevation	Core i5-1235U	Intel	x86-64	Alder Lake	2022
Artemisia	2× Xeon Gold 5420+	Intel	x86-64	Sapphire Rapids	2023
Descent	i7-1370P	Intel	x86-64	Raptor Lake	2023

Table 1: Lab-machine taxonomy sorted by processor-generation or platform introduction year; citations identify the primary vendor sources.

Machine	SGX Active	Hybrid
Sunbird	No	No
Charnwood	No	No
Ookay	No	No
Crux	No	No
Skylark	No	No
Elevation	No	Yes
Artemisia	Yes	No
Descent	No	Yes

Table 2: Lab-machine taxonomy defining which machines have SGX active, hybrid, or neither.

3 Methodology

All experiment were ran three times and summarized with the average of each runs output. Multiple runs were used to limit outliers and produce consistent results. All experiements referenced the header file below, listing 1, to conduct each experiment. This header file can be found in the repository at `CSC591-Project3▸common▸include▸bench_common.h`. Once each of the three runs concluded, the data was then summarized and organzied into a csv file to read data, and figures to visualize the data. This data was compiled using various python scripts which can be found in `CSC591-Project3▸scripts▸collect.py` and `CSC591-Project3▸scripts▸summarize.py`

Listing 1: bench_common.h

```
// bench_common.h - timing / pinning / flushing helpers shared by the C exercises.
// Supports x86-64 and AArch64 (Linux, GCC/Clang). One header, no per-machine scripts.
// Timing only: a cycle/timer counter and clock_gettime. No performance-monitoring counters (course requirement R1).
//
//   x86-64 : tsc_*  = TSC (lfence/rdtsc, rdtscp/lfence); clflush; mfence
//   AArch64: tsc_*  = generic timer cntvct_el0 (+isb). Its frequency (cntfrq_el0, typically 25-50 MHz) is far below the CPU
//           clock, so ticks are coarse: use the ns columns (clock_gettime) for results. Flush = dc civac; fence = dsb sy.
#ifndef BENCH_COMMON_H
#define BENCH_COMMON_H
#define _GNU_SOURCE
#include <sched.h>
#include <stdint.h>
#include <stdio.h>
#include <time.h>
#include <unistd.h>

static inline double now_ns_fwd(void);   // defined below
#if defined(__x86_64__)
#define BENCH_ARCH "x86-64"
static inline uint64_t tsc_begin(void) {
    uint32_t lo, hi;
    __asm__ volatile("lfence\n\trdtsc" : "=a"(lo), "=d"(hi)::"memory");
    return ((uint64_t)hi << 32) | lo;
}
static inline uint64_t tsc_end(void) {
    uint32_t lo, hi;
    __asm__ volatile("rdtscp\n\tlfence" : "=a"(lo), "=d"(hi)::"rcx", "memory");
    return ((uint64_t)hi << 32) | lo;
}
static inline void clflush_line(const void *p) { __asm__ volatile("clflush␣(%0)" ::"r"(p) : "memory"); }
static inline void mfence(void) { __asm__ volatile("mfence" ::: "memory"); }
static inline double timer_ghz_nominal(void) { return 0.0; }   // x86: calibrate against clock_gettime (tsc_ghz below)

// Core clock estimate in GHz WITHOUT performance counters: a chain of dependent `add %rbx,%rax` (register source) runs at
// exactly one add per cycle on x86 cores (latency 1), so  GHz = adds / ns.  Best of 5 runs (the fastest sees the real clock).
// NOTE: do NOT use `add $imm,%reg` chains for this: Golden Cove-class cores fold dependent immediate adds at the rename
// stage and execute them at several per cycle (measured here: ~3x too fast), which would overstate the clock.
static inline double core_ghz(void) {
    const uint64_t iters = 1000000;           // x 16 adds = 16M dependent adds per run
    double best = 0;
    for (int r = 0; r < 5; r++) {
        double t0 = now_ns_fwd();
        __asm__ volatile("mov␣$1,␣%%rbx\n\t"
                         "mov␣%0,␣%%r15\n\t"
                         "1:\n\t"
                         ".rept␣16\n\t␣add␣%%rbx,␣%%rax\n\t␣.endr\n\t"
                         "dec␣%%r15\n\t"
                         "jnz␣1b\n\t" ::"r"(iters) : "rax", "rbx", "r15", "cc");
        double g = (double)(iters * 16) / (now_ns_fwd() - t0);
        if (g > best) best = g;
    }
    return best;
}
#elif defined(__aarch64__)
#define BENCH_ARCH "aarch64"
static inline uint64_t tsc_begin(void) {
    uint64_t v;
    __asm__ volatile("isb\n\tmrs␣%0,␣cntvct_el0" : "=r"(v)::"memory");
    return v;
}
static inline uint64_t tsc_end(void) {
    uint64_t v;
    __asm__ volatile("mrs␣%0,␣cntvct_el0\n\tisb" : "=r"(v)::"memory");
    return v;
}
static inline void clflush_line(const void *p) { __asm__ volatile("dc␣civac,␣%0" ::"r"(p) : "memory"); }
static inline void mfence(void) { __asm__ volatile("dsb␣sy" ::: "memory"); }
static inline double core_ghz(void) { return 0.0; }            // not implemented on AArch64
static inline double timer_ghz_nominal(void) {                 // generic timer frequency in GHz (e.g. 0.025 for 25 MHz)
    uint64_t f;
    __asm__ volatile("mrs␣%0,␣cntfrq_el0" : "=r"(f));
    return (double)f / 1e9;
}
```

```
#else
#error "bench_common.h␣supports␣x86-64␣and␣AArch64␣only"
#endif

static inline double now_ns(void) {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec * 1e9 + ts.tv_nsec;
}
static inline double now_ns_fwd(void) { return now_ns(); }
static inline int pin_to_cpu(int cpu) {   // returns 0 on success
    cpu_set_t set;
    CPU_ZERO(&set);
    CPU_SET(cpu, &set);
    return sched_setaffinity(0, sizeof(set), &set);
}
static inline uint64_t xorshift64(uint64_t *state) {
    uint64_t x = *state;
    x ^= x << 13; x ^= x >> 7; x ^= x << 17;
    return *state = x;
}
static inline double tsc_ghz(void) {      // timer ticks per ns
#if defined(__aarch64__)
    return timer_ghz_nominal();
#else
    double t0 = now_ns(); uint64_t c0 = tsc_begin();
    usleep(200000);
    uint64_t c1 = tsc_end(); double t1 = now_ns();
    return (double)(c1 - c0) / (t1 - t0);
#endif
}
#endif
```

4 ROB Size

The first experiment conducted was measuring the Reorder Buffer (ROB) size of each machine. Each experiment was ran three times. At the end of the three test runs, data was collected and summarized into a table. The CSV summary table, Table 3, can be found in the repository at `CSC591-Project3‣report‣tables‣ex04_rob_summary.csv`. The ROB Size graph, Figure 1, can be found in the repository at `CSC591-Project3‣report‣figures‣ex04_rob_overlay.png`

Machine	Core	Median ROB	Low Latency (ns)	Hi/Low Ratio
artemisia	cpu29	498.0	166.3	1.74
charnwood	cpu2	223.0	112.9	1.81
crux	cpu3	222.0	90.1	1.8
descent	ecore	253.0	138.1	1.8
descent	pcore	512.0	120.8	1.85
elevation	ecore	252.0	152.5	1.62
elevation	pcore	512.0	115.1	1.75
ookay	cpu2	222.0	87.7	1.85
skylark	cpu2	223.0	151.6	1.8
sunbird	cpu8	190.0	132.4	1.63

Table 3: ROB Size Data Across Machines

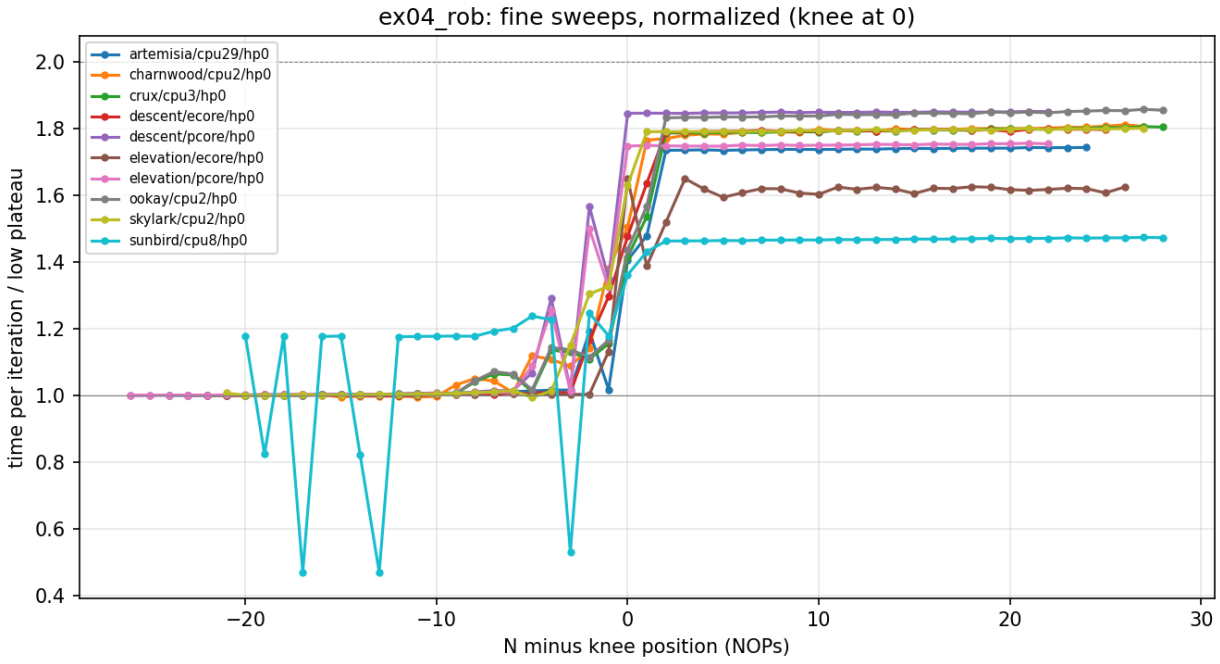


Figure 1: ROB Size Graph Across All Machines

Based on the data and graph, the P-cores in the hybrid machines of Elevation and Descent have the highest ROB capacity, nearly twice the size of their E-cores. However, the E-cores ROB size slightly larger, if not similar, to the size of non-SGX processors. Sunbird has the lowest ROB capacity, which is nearly half the size than the SGX enabled machine, Artemisia. Overall, Artemisia

has a higher ROB capacity compared to the non SGX enabled machines. However, Artemisia did have the largest latency among the other processors. This finding presents an interesting claim that SGX enabled and hybrid processors have a larger ROB capacity than non-SGX and non-hybird processors.

Each of the runs' data, logs, and output can be found in each of the machines' respective folder following the format `CSC591-Project3‣data‣<machine_name>‣ex04_rob‣<run_folder>`.

Below, Listing 2, is the main code snippet used to analyze the ROB size of each machine. This can be found in the repository at `CSC591-Project3‣exercises‣ex04_rob‣rob_test.c`

Listing 2: ROB Size Estimate Code Block

```
// rob_test.c - Reorder-buffer (ROB) size microbenchmark for x86-64 / Linux / GCC or Clang.
//
// IDEA
//   Each loop iteration issues two DRAM-missing loads, A and B, separated by N one-byte NOPs:
//
//       A: load (miss)  -> N x nop -> B: load (miss)
//
//   * If the ROB can hold A, the N NOPs and B (N + 3 entries), B enters the window while A is still
//     outstanding, so both misses overlap:               time/iter ~ 1 x DRAM latency.
//   * If N + 3 > ROB size, B cannot be allocated until A retires, so the misses serialize:
//                                                       time/iter ~ 2 x DRAM latency.
//   The knee of time/iter vs N gives the ROB size:  ROB ~ N_knee + 2  (N_knee = first "slow" N).
//
//   Iterations are chained through a data dependence (the address of A and B in iteration i+1 depends
//   on the value loaded by B in iteration i; the buffer holds zeros, so the address is unchanged).
//   That stops the out-of-order window from overlapping *different* iterations.
//
//   The NOP count is varied WITHOUT JIT: the asm contains a sled of MAX_NOPS NOPs and an indirect jump
//   enters the sled N instructions before its end. (No RWX memory needed -> portable to SGX enclaves.)
//   Written for x86-64 and AArch64 in this one file; the compiler selects the matching kernel.
//
// OUTPUT (stdout, CSV):  nops,rep,tsc_per_iter,ns_per_iter      ('#' lines are metadata)

#include "bench_common.h"   // timing, pinning, flushing helpers for x86-64 and AArch64 (common/include)
#include <getopt.h>
#include <stdlib.h>
#include <string.h>
#include <sys/mman.h>

#define XSTR(x) #x
#define STR(x) XSTR(x)
#define MAX_NOPS 1024
#define LINE 64UL
#define HUGE_SZ (2UL << 20)

// ---------- the kernel ----------
// offs : 2*iters byte-offsets (multiples of 64) into base; offs[2i] -> load A, offs[2i+1] -> load B
// base : buffer of zeros, much larger than the LLC
// nops : number of filler instructions between A and B (0..MAX_NOPS)
// The same experiment is written twice, once per ISA; the compiler picks the right one. Everything else is shared.
#if defined(__x86_64__)
__attribute__((noinline)) static void rob_kernel(const uint64_t *offs, const char *base,
                                                 uint64_t iters, uint64_t nops) {
    __asm__ volatile(
        "xor␣␣␣␣%%r9d,␣%%r9d␣␣␣␣␣␣␣␣␣␣␣␣␣\n\t" // r9 = value loaded by previous B (always 0)
        "lea␣␣␣␣3f(%%rip),␣%%r10␣␣␣␣␣␣␣␣␣\n\t" // r10 = end of NOP sled
        "sub␣␣␣␣%[n],␣%%r10␣␣␣␣␣␣␣␣␣␣␣␣␣\n\t" // r10 = entry point: exactly n NOPs (1 byte each) before label 3
        "1:␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣\n\t"
        "mov␣␣␣␣(%%rdi),␣%%rax␣␣␣␣␣␣␣␣␣␣␣\n\t" // offset for A
        "mov␣␣␣␣8(%%rdi),␣%%rcx␣␣␣␣␣␣␣␣␣␣\n\t" // offset for B
        "add␣␣␣␣%%r9,␣%%rax␣␣␣␣␣␣␣␣␣␣␣␣␣\n\t" // A address depends on previous B (adds 0)
        "add␣␣␣␣%%r9,␣%%rcx␣␣␣␣␣␣␣␣␣␣␣␣␣\n\t" // B address depends on previous B (adds 0)
        "mov␣␣␣␣(%%rsi,%%rax),␣%%r8␣␣␣␣␣\n\t" // load A  (DRAM miss)
        "jmp␣␣␣␣*%%r10␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣\n\t" // enter NOP sled
        ".rept␣" STR(MAX_NOPS) "␣␣␣␣␣␣␣␣␣\n\t"
        "nop␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣\n\t" // 1-byte NOPs (0x90)
        ".endr␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣\n\t"
        "3:␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣\n\t"
        "mov␣␣␣␣(%%rsi,%%rcx),␣%%r9␣␣␣␣␣\n\t" // load B  (DRAM miss)
        "add␣␣␣␣$16,␣%%rdi␣␣␣␣␣␣␣␣␣␣␣␣␣\n\t"
        "dec␣␣␣␣%%rdx␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣\n\t"
        "jnz␣␣␣␣1b␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣\n\t"
        : "+D"(offs), "+d"(iters)
        : "S"(base), [n] "r"(nops)
        : "rax", "rcx", "r8", "r9", "r10", "cc", "memory");
}
#elif defined(__aarch64__)
// Every A64 instruction is 4 bytes, so the entry point is 4*n bytes before the end of the sled.
// Filler instruction: 'nop' by default. If a core drops NOPs before the ROB, no knee appears; then rebuild with the
// independent ALU filler:   make CFLAGS_EXTRA=-DROB_FILLER_ALU     (via fleet.sh: MAKE_ARGS="CFLAGS_EXTRA=-DROB_FILLER_ALU")
// and discuss in the report that ALU fillers also occupy scheduler entries, so that knee may be lower than the ROB size.
#ifndef FILLER
#  ifdef ROB_FILLER_ALU
#    define FILLER "add␣x11,␣xzr,␣xzr"
```

```
#  else
#    define FILLER "nop"
#  endif
#endif
__attribute__((noinline)) static void rob_kernel(const uint64_t *offs, const char *base,
                                                 uint64_t iters, uint64_t nops) {
    __asm__ volatile(
        "mov␣␣␣x9,␣xzr␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣\n\t" // x9 = value loaded by previous B (always 0)
        "adr␣␣␣x10,␣3f␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣\n\t" // x10 = end of filler sled
        "sub␣␣␣x10,␣x10,␣%[n],␣lsl␣#2␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣\n\t" // x10 = entry point: n instructions (4 bytes each) before label 3
        "1:␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣
```

```
            double n0 = now_ns();
            uint64_t c0 = tsc_begin();
            rob_kernel(offs, buf, iters, (uint64_t)n);
            uint64_t c1 = tsc_end();
            double n1 = now_ns();

            if (r >= 0) printf("%d,%d,%.2f,%.2f\n", n, r, (double)(c1 - c0) / iters, (n1 - n0) / iters);
        }
        fflush(stdout);
    }
    return 0;
}
```

5 Processor Pipelining

The second experiment conducted was verifying if a processor was pipelined or not. Each experiment was ran three times. At the end of the three test runs, data was collected and summarized into a table. The CSV summary table, Table 4, can be found in the repository at `CSC591-Project3 ‣ report ‣ tables ‣ ex01_pipelined_summary`. The figures shown below of the each of the machine's data is representation of the last test ran to determine if the processor is pipelined. Each of the machines respective figures can be found in the repository at `CSC591-Project3 ‣ data ‣ <machine_name> ‣ ex01_pipelined ‣ <run_number> ‣ plot.png`

Machine	Core	ADD IMM Fold Factor	ADD Peak IPC
artemisia	cpu29	5.33	4.975
charnwood	cpu2	1.0	3.98
crux	cpu3	1.0	3.98
descent	ecore	1.0	3.97
descent	pcore	5.31	4.97
elevation	ecore	1.0	3.97
elevation	pcore	5.41	5.05
ookay	cpu2	1.01	4.01
skylark	cpu2	1.02	4.03
sunbird	cpu8	1.0	3.98

Table 4: Processor Pipelining Summary

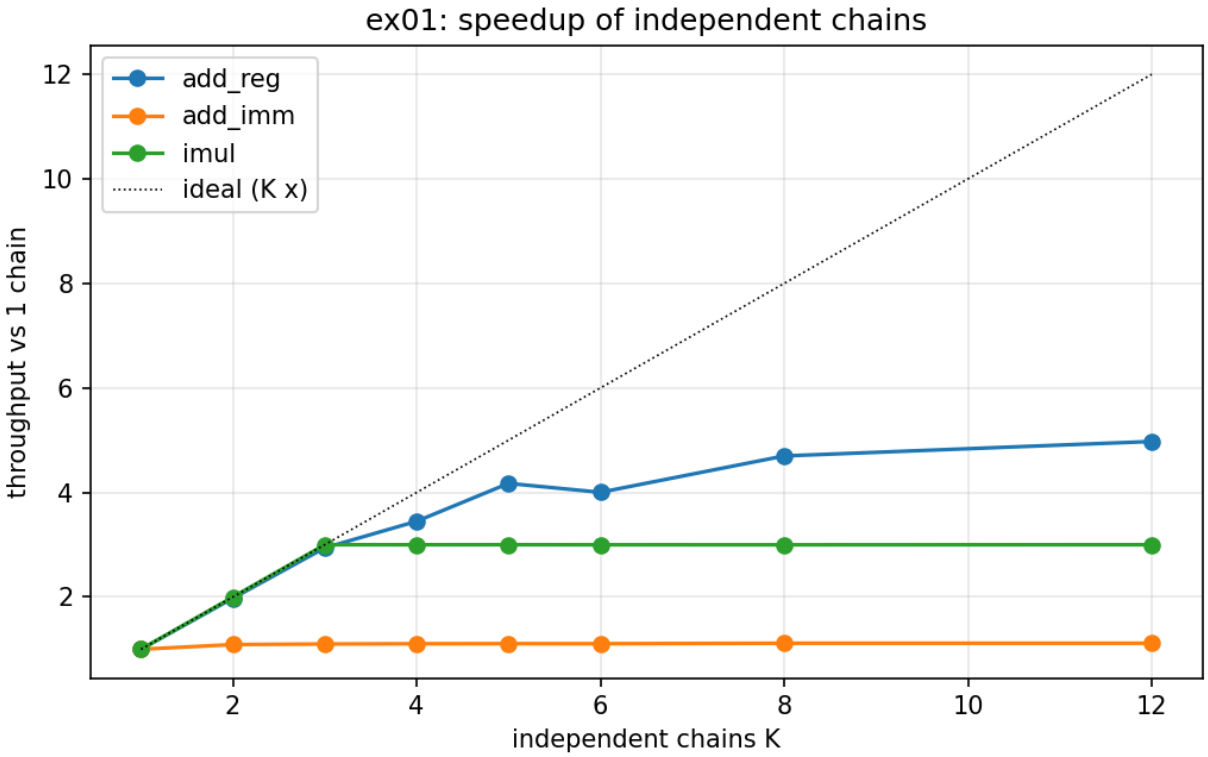


Figure 2: Artemisia Pipelining Data

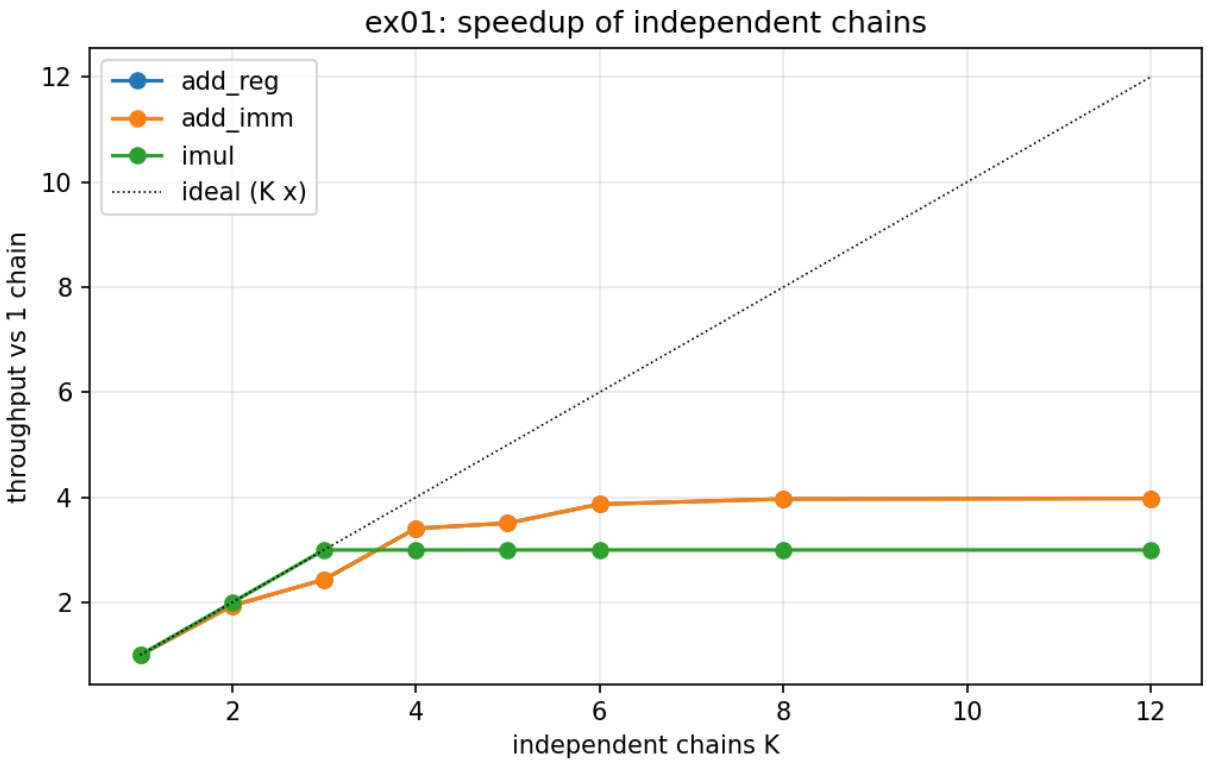


Figure 3: Charnwood Pipelining Data

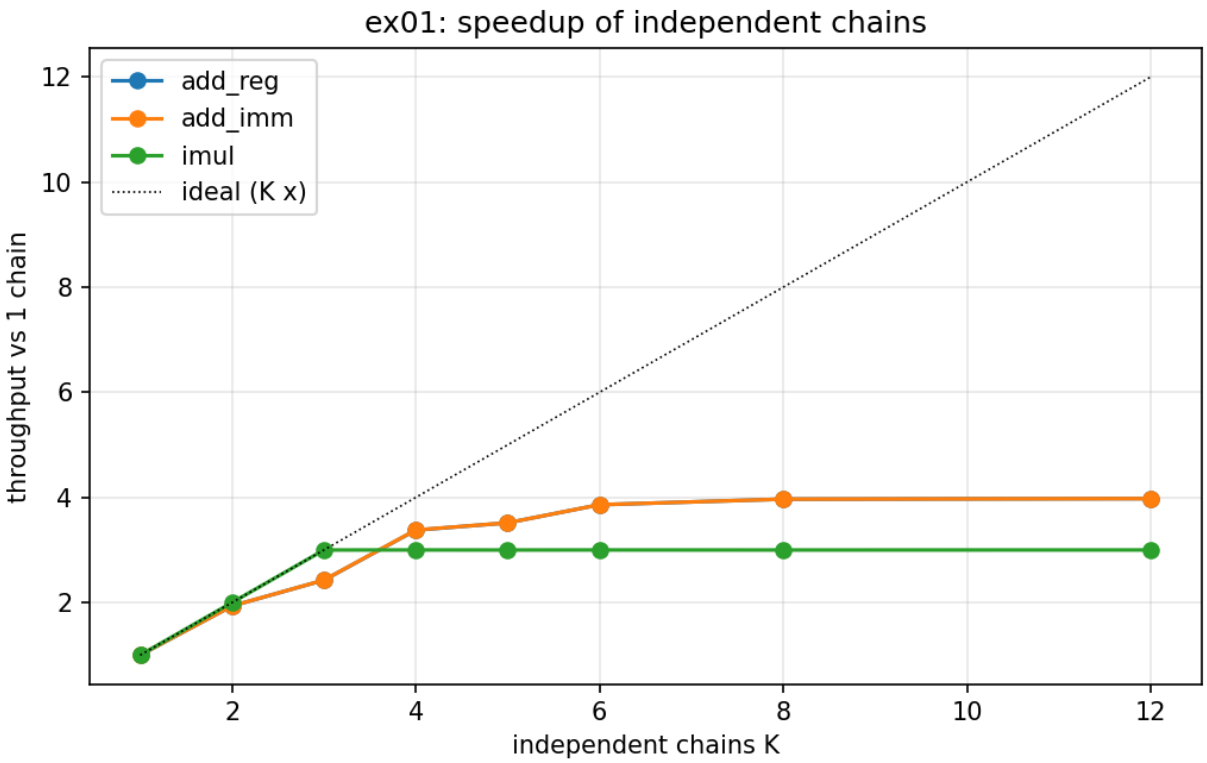


Figure 4: Crux Pipelining Data

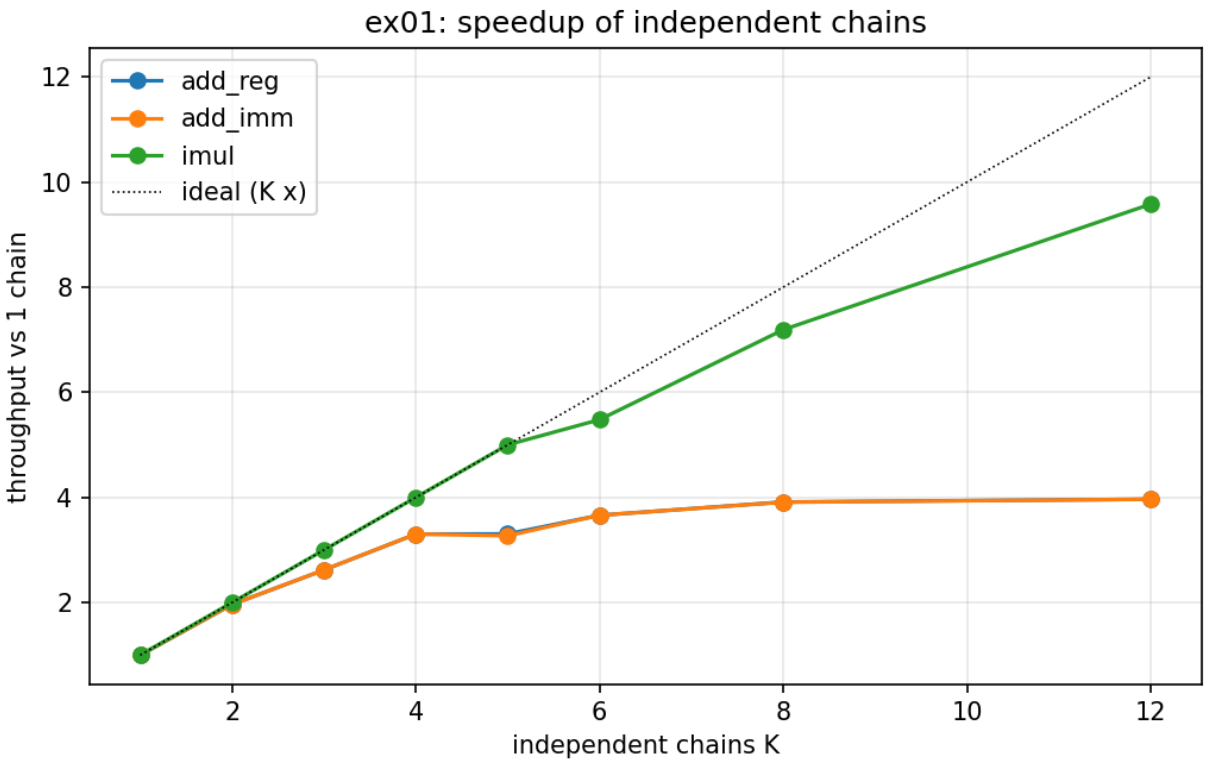


Figure 5: Descent E-core Pipelining Data

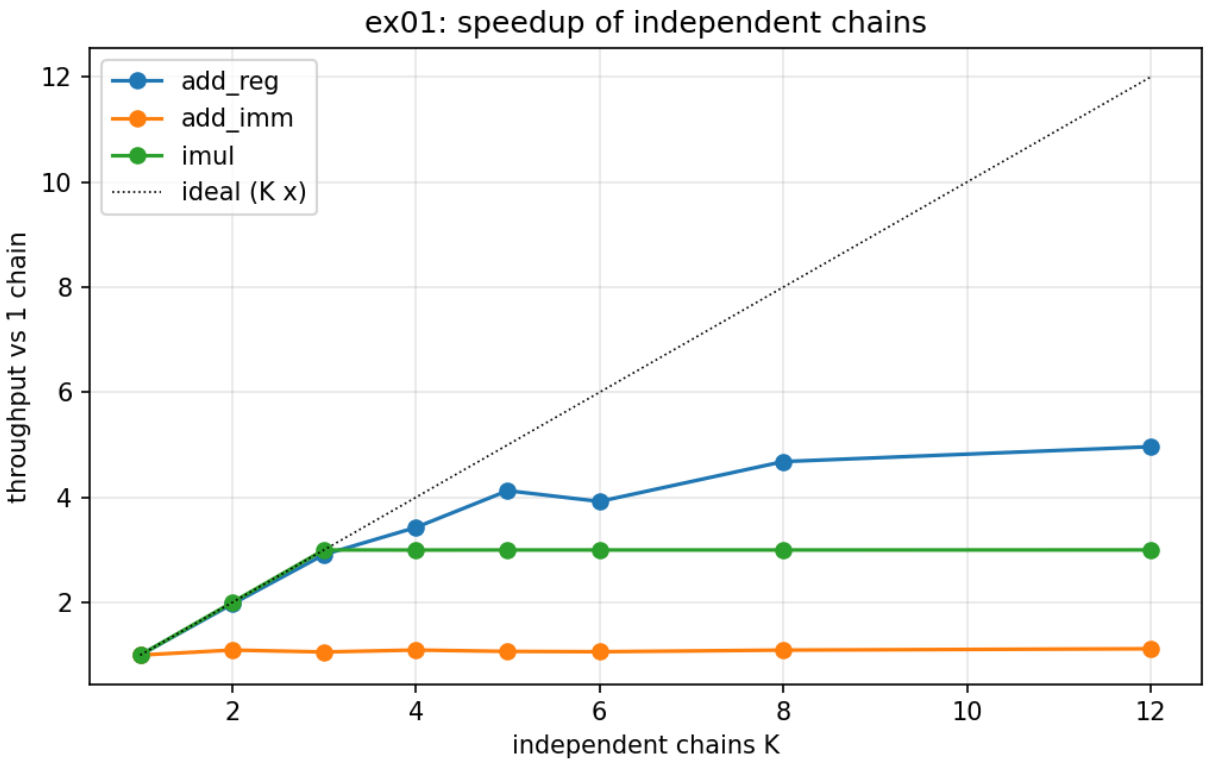


Figure 6: Descent P-core Pipelining Data

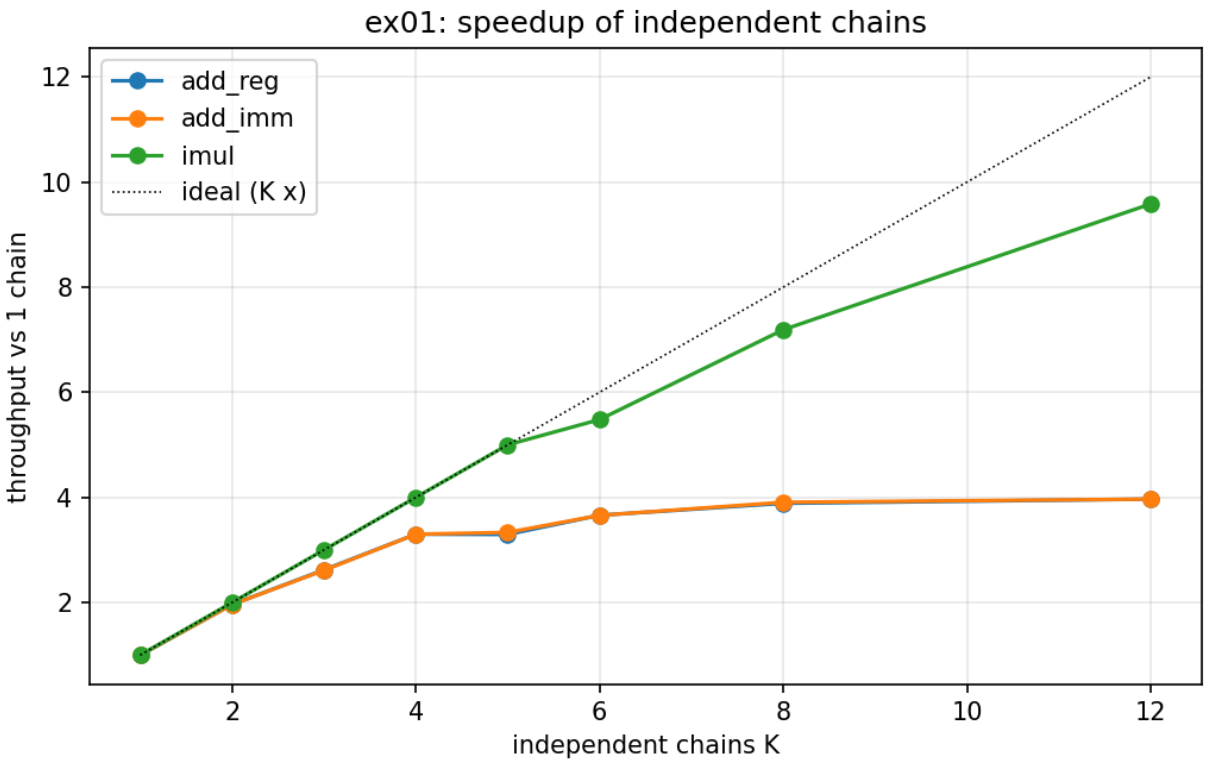


Figure 7: Elevation E-core Pipelining Data

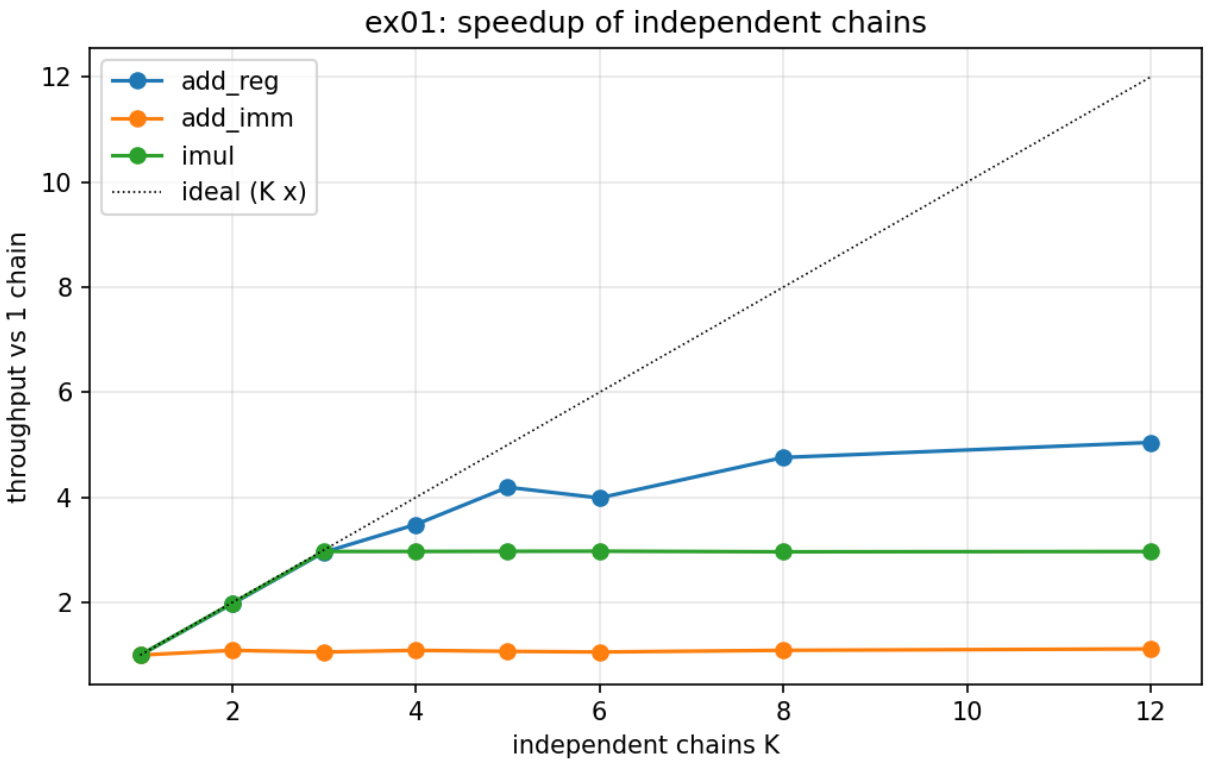


Figure 8: Elevation P-core Pipelining Data

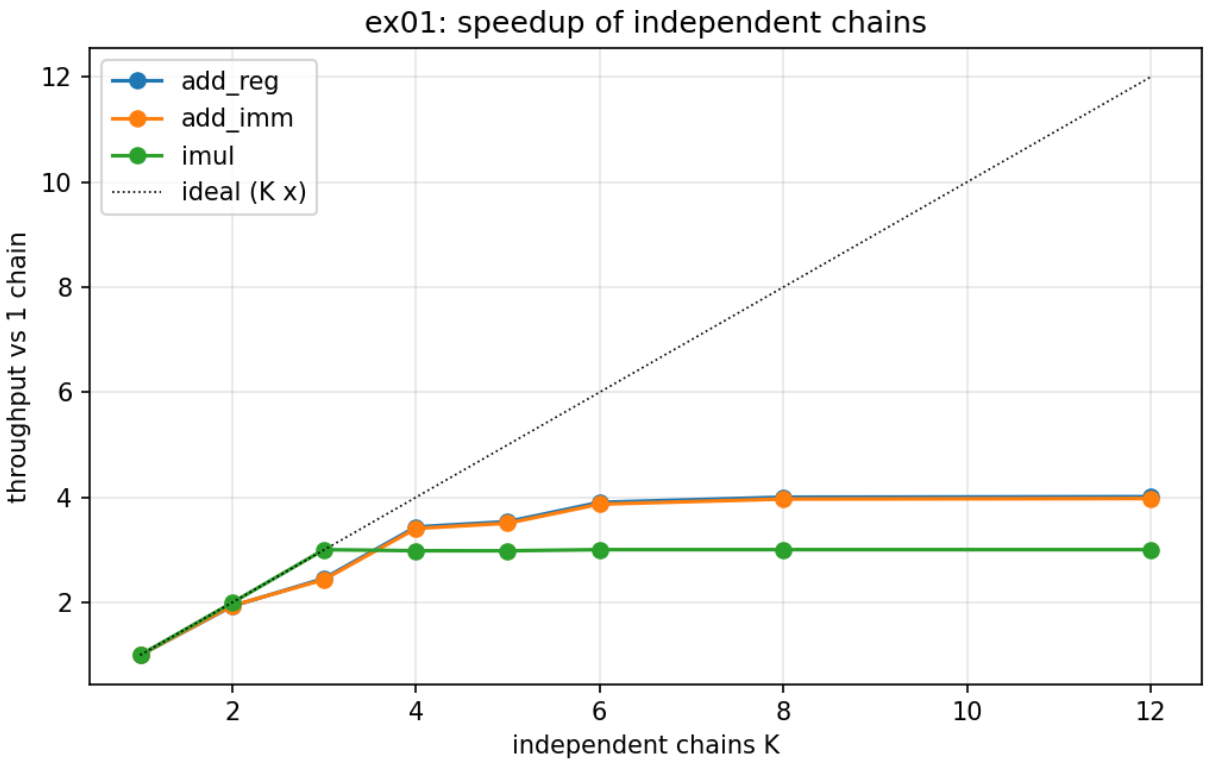


Figure 9: Ookay Pipelining Data

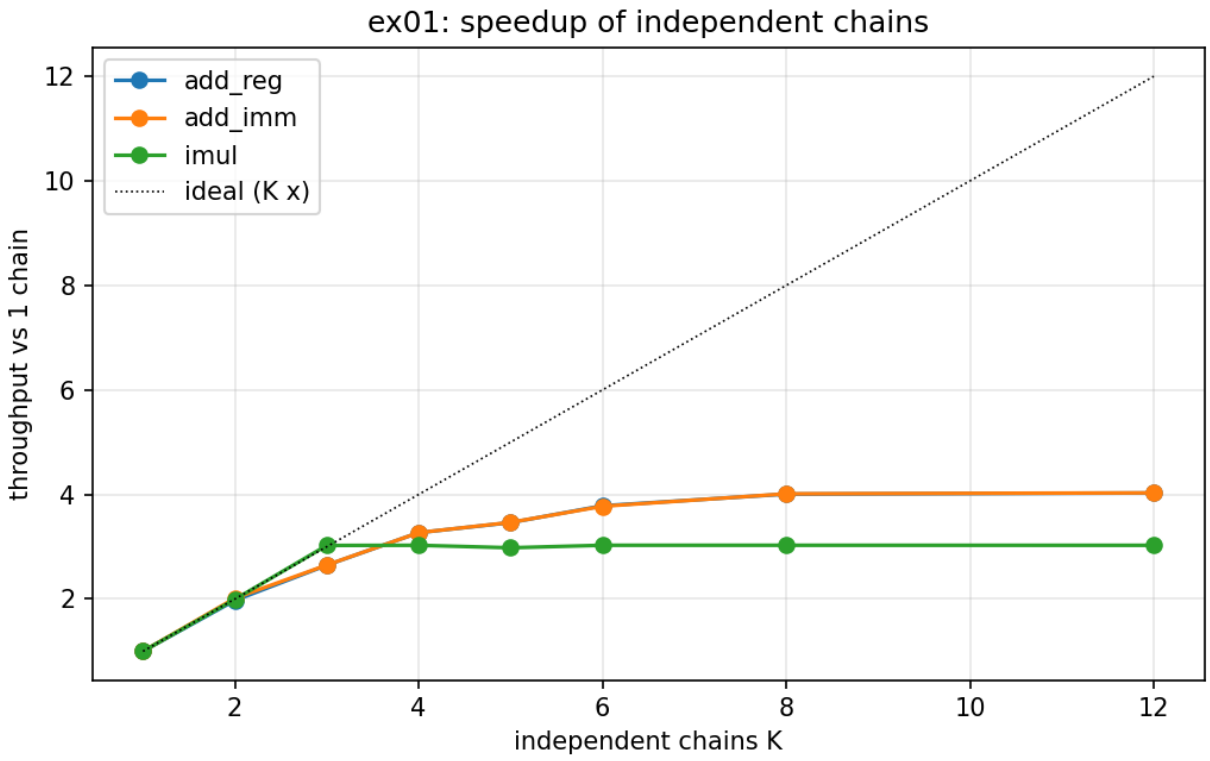


Figure 10: Skylark Pipelining Data

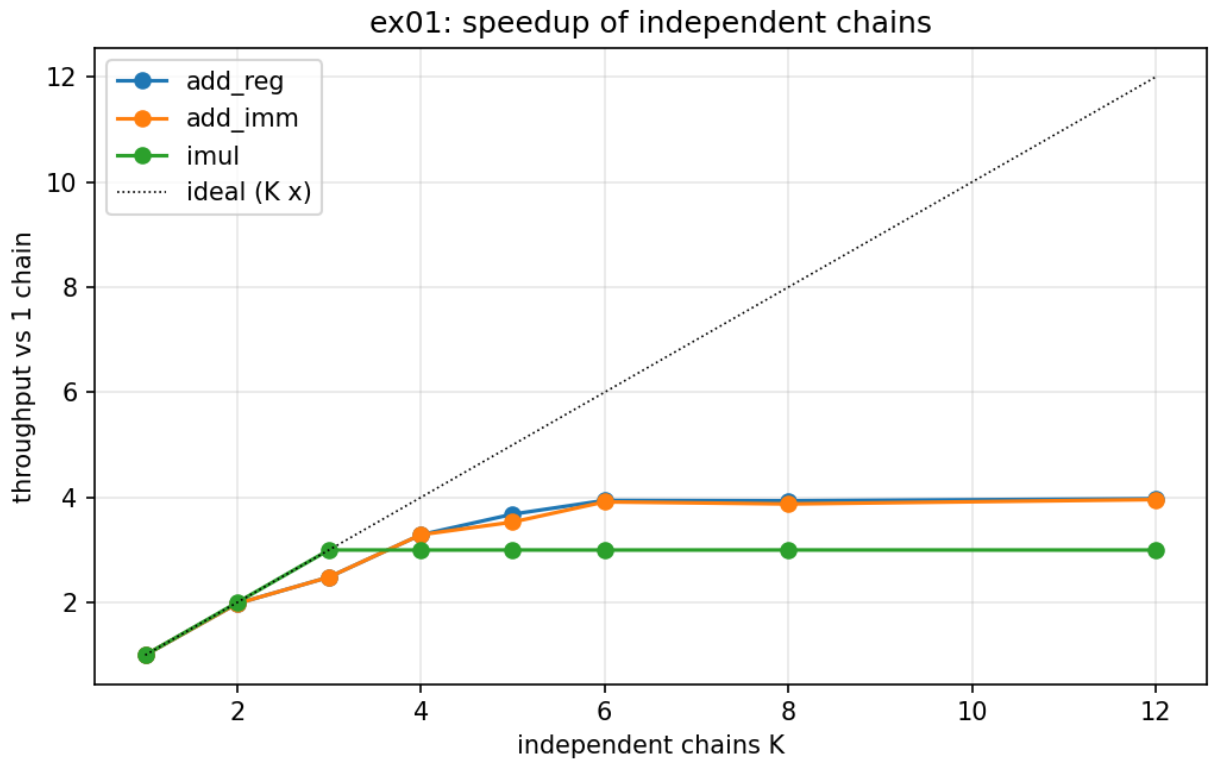


Figure 11: Sunbird Pipelining Data

Based on the figures, the e-cores of the hybrid machines, Descent and Elevation, showed more of a linear growth in throughput as independent chains increased compared to the other machines which would plateau in throughput when K independent chains was around three. In the P-cores and Artemisia, the throughput of regular ADD operations was higher than ADD immediate value operations, whereas in the other machines, the throughput was the exact same. The P-cores and Artemisia also had a much higher ADD IMM Fold Factor, with a value more than five times the other machines. The data shows that all processors were infact pipelined. However, a claim can be made that SGX and hybrid machines can optimize instructions more effectively by folding operations into other execution instructions; thus, SGX-enabled and hybrid processors are more efficient in pipeline arithmetic instructions.

Below, Listing 3, is the main code snippet used to test the pipelining of each machine. This can be found in the repository at `CSC591-Project3‣exercises‣ex01_pipelined‣pipe_test.c`

Listing 3: Pipelining Code Block

```
// pipe_test.c - Is the processor pipelined? (x86-64, Linux, GCC/Clang)
//
// IDEA. A dependent chain of N instructions takes N x latency. If the unit is PIPELINED, several INDEPENDENT chains
// proceed at the same time: K chains finish in about the same time as one, so throughput grows ~K-fold until a hardware
// limit (the unit's latency/throughput ratio, or the machine width).
//   op add_reg : add %r14,%reg   register-source add, latency 1 -> the "one instruction per cycle" reference
//   op add_imm : add $1,%reg     immediate add (newer Intel cores fold dependent ones at rename: faster than 1/cycle)
//   op imul    : imul $3,%reg,%reg   3-cycle latency, 1/cycle throughput on x86: speedup saturating near K=3 is the
//                                    classic signature of a PIPELINED multiplier (new multiply starts every cycle)
// Ratios of throughputs are independent of the core clock, so no counters and no frequency assumptions are needed.
//
// OUTPUT (CSV on stdout): op,k,rep,ns,instrs      ('#' lines = metadata)

#include "bench_common.h"
#include <getopt.h>
#include <stdlib.h>
#include <string.h>
```

```
#if !defined(__x86_64__)
#error "pipe_test.c␣is␣x86-64␣only"
#endif

#define XSTR(x) #x
#define STR(x) XSTR(x)
#define GROUPS 16                // repetitions of the K-instruction group per loop iteration (amortizes the loop branch)

#define OP_add_reg(r) "add␣%%r14,␣%%" #r "\n\t"
#define OP_add_imm(r) "add␣$1,␣%%" #r "\n\t"
#define OP_imul(r) "imul␣$3,␣%%" #r ",␣%%" #r "\n\t"

// K independent accumulator chains = K different registers (r14 = constant source, r15 = loop counter)
#define B1(O) O(rax)
#define B2(O) B1(O) O(rbx)
#define B3(O) B2(O) O(rcx)
#define B4(O) B3(O) O(rdx)
#define B5(O) B4(O) O(rsi)
#define B6(O) B5(O) O(rdi)
#define B8(O) B6(O) O(r8) O(r9)
#define B12(O) B8(O) O(r10) O(r11) O(r12) O(r13)

#define KERNEL(name, O, B)                                                                          \
    static void name(uint64_t n) {                                                                  \
        __asm__ volatile("mov␣$1,␣%%r14\n\t"                                                        \
                         "mov␣%[n],␣%%r15\n\t"                                                      \
                         "1:\n\t"                                                                   \
                         ".rept␣" STR(GROUPS) "\n\t" B(O) ".endr\n\t"                               \
                         "dec␣%%r15\n\t"                                                            \
                         "jnz␣1b\n\t"                                                               \
                         :                                                                          \
                         : [n] "m"(n)                                                               \
                         : "rax", "rbx", "rcx", "rdx", "rsi", "rdi", "r8", "r9", "r10", "r11", "r12", "r13", "r14", \
                           "r15", "cc");                                                            \
    }

#define ALLK(op) KERNEL(k_##op##_1, OP_##op, B1) KERNEL(k_##op##_2, OP_##op, B2) KERNEL(k_##op##_3, OP_##op, B3) \
    KERNEL(k_##op##_4, OP_##op, B4) KERNEL(k_##op##_5, OP_##op, B5) KERNEL(k_##op##_6, OP_##op, B6)             \
    KERNEL(k_##op##_8, OP_##op, B8) KERNEL(k_##op##_12, OP_##op, B12)
ALLK(add_reg)
ALLK(add_imm)
ALLK(imul)

typedef void (*kfn)(uint64_t);
struct kern { const char *op; int k; kfn fn; };
#define ENT(op, K) { #op, K, k_##op##_##K }
#define ROW(op) ENT(op, 1), ENT(op, 2), ENT(op, 3), ENT(op, 4), ENT(op, 5), ENT(op, 6), ENT(op, 8), ENT(op, 12)
static const struct kern table[] = { ROW(add_reg), ROW(add_imm), ROW(imul) };
#define NK (int)(sizeof(table) / sizeof(table[0]))

int main(int argc, char **argv) {
    int core = -1, reps = 9, c;
    uint64_t total = 16000000;           // instructions per timed run (about 5-20 ms)
    while ((c = getopt(argc, argv, "c:r:n:h")) != -1) {
        if (c == 'c') core = atoi(optarg);
        else if (c == 'r') reps = atoi(optarg);
        else if (c == 'n') total = strtoull(optarg, 0, 10);
        else { fprintf(stderr, "usage:␣%s␣[-c␣core]␣[-r␣reps]␣[-n␣instructions_per_run]\n", argv[0]); return 1; }
    }
    if (core >= 0 && pin_to_cpu(core)) { perror("sched_setaffinity"); return 1; }

    printf("#␣arch=%s␣core=%d␣reps=%d␣instrs_per_run=%lu␣core_ghz=%.4f\n", BENCH_ARCH, core, reps, (unsigned long)total, core_ghz());
    printf("op,k,rep,ns,instrs\n");
    // rep-major: a burst of external noise hits every kernel once instead of one kernel for all reps
    for (int r = -1; r < reps; r++) {          // r = -1: warm-up pass
        for (int i = 0; i < NK; i++) {
            uint64_t iters = total / (GROUPS * table[i].k);
            double t0 = now_ns();
            table[i].fn(iters);
            double t1 = now_ns();
            if (r >= 0) printf("%s,%d,%d,%.1f,%lu\n", table[i].op, table[i].k, r, t1 - t0, (unsigned long)(iters * GROUPS * table[i].k));
        }
        fflush(stdout);
    }
    return 0;
}
```

6 Out-of-Order Execution

The third experiment conducted was verifying if a processor used out-of-order execution. Each experiment was ran three times. At the end of the three test runs, data was collected and summarized into a table. The CSV summary table, Table 5, can be found in the repository at `CSC591-Project3 ▸ report ▸ tables ▸ ex02_ooo_summary`. The figures shown below of the each of the machine's data is representation of the last test ran to determine if the processor used out-of-order execution. Each of the machines respective figures can be found in the repository at `CSC591-Project3 ▸ data ▸ <machine_name> ▸ ex02_ooo ▸ <run_number> ▸ plot.png`

Machine	Core	Dep. Cost (ns)	Dep. Cycles/Filler	Indep. Cost (ns)	Indep. Rise Factor	Miss Latency (ns)
artemisia	cpu29	47.6	1.0	0.0	240.0	139.9
charnwood	cpu2	48.1	1.42	4.2	160.0	96.2
crux	cpu3	29.1	1.06	1.3	176.0	79.6
descent	ecore	34.7	1.05	-0.2	240.0	110.5
descent	pcore	25.2	0.98	0.1	272.0	99.2
elevation	ecore	41.5	1.06	-0.6	224.0	118.5
elevation	pcore	31.1	1.06	-0.0	256.0	92.8
ookay	cpu2	32.6	1.07	2.0	176.0	79.3
skylark	cpu2	39.4	1.01	2.8	192.0	135.6
sunbird	cpu8	52.2	1.02	8.4	144.0	105.3

Table 5: Processor Out-of-Order Execution Summary

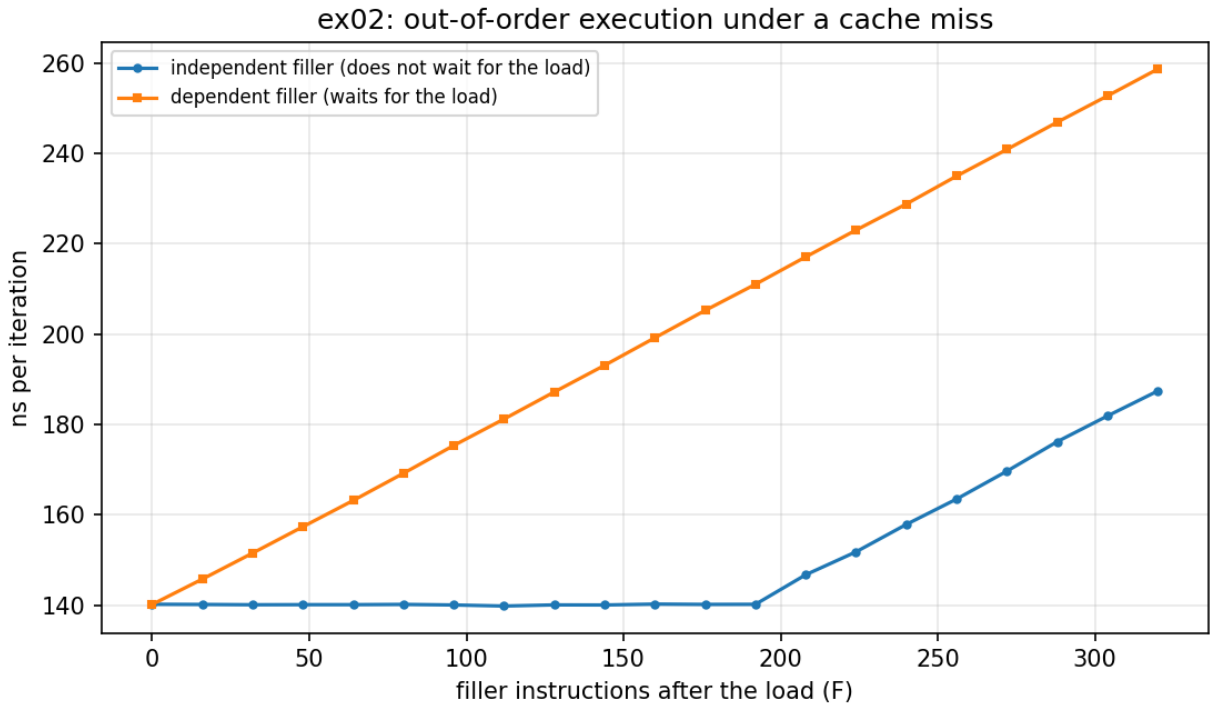


Figure 12: Artemisia OOO Execution Data

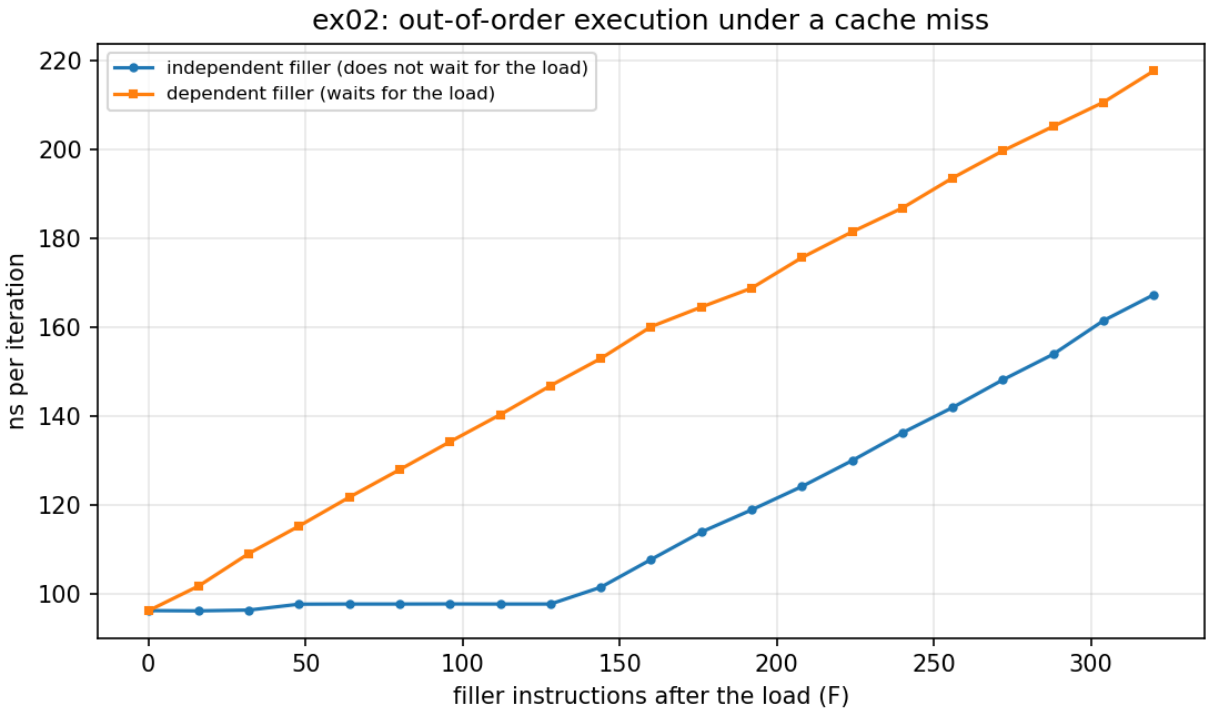


Figure 13: Charnwood OOO Execution Data

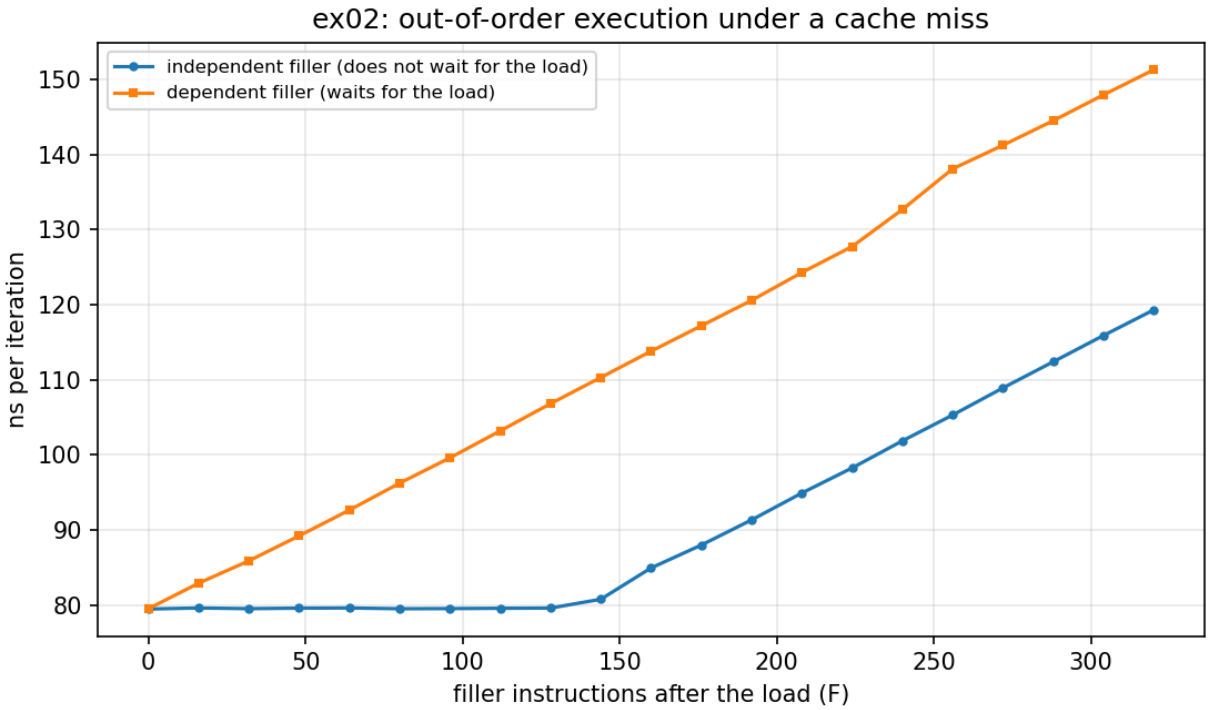


Figure 14: Crux OOO Execution Data

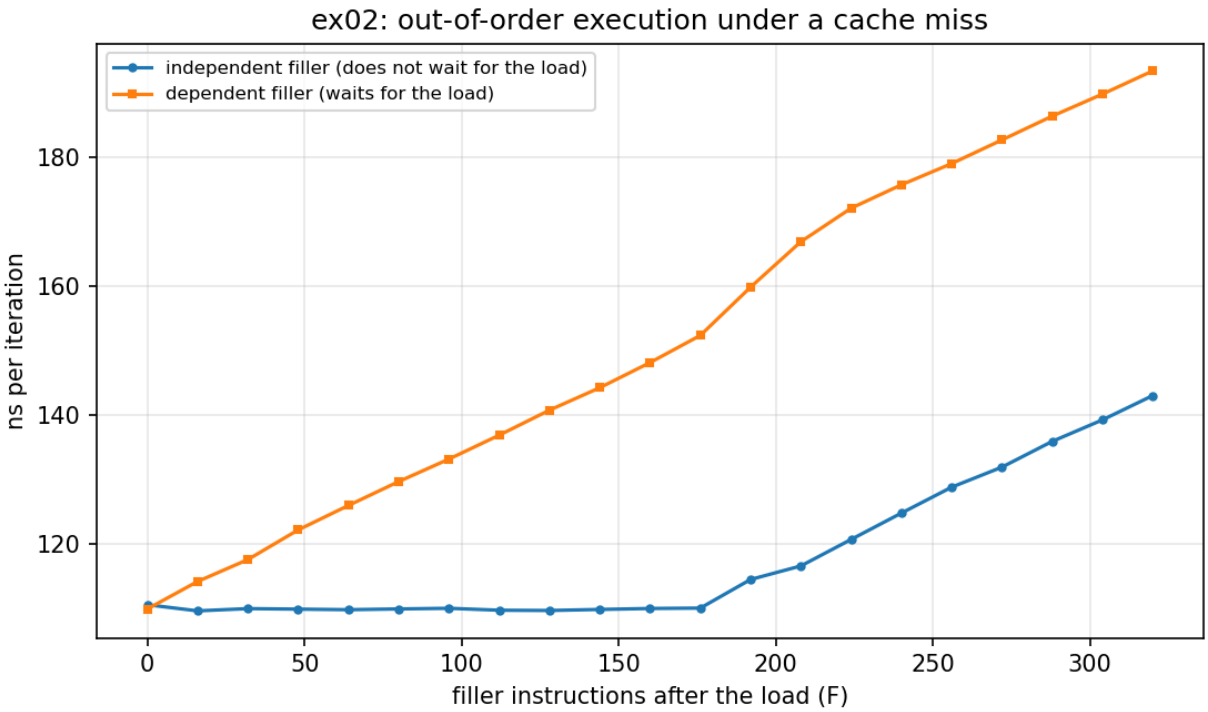


Figure 15: Descent E-core OOO Execution Data

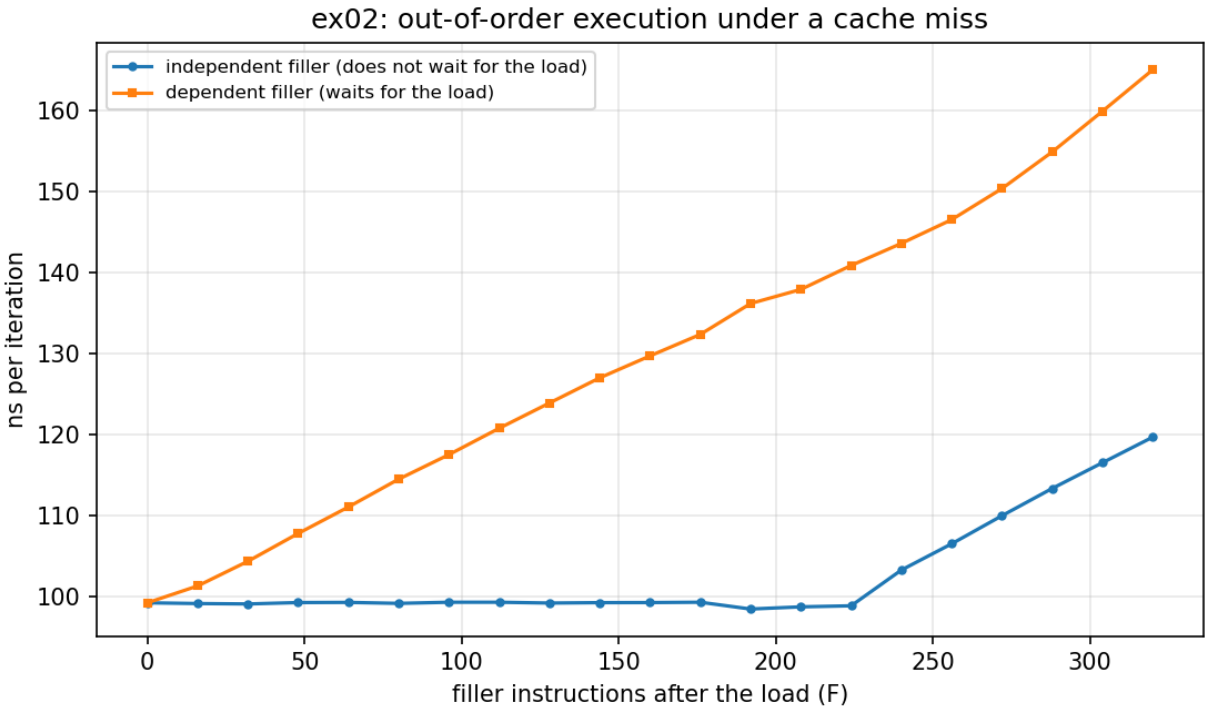


Figure 16: Descent P-core OOO Execution Data

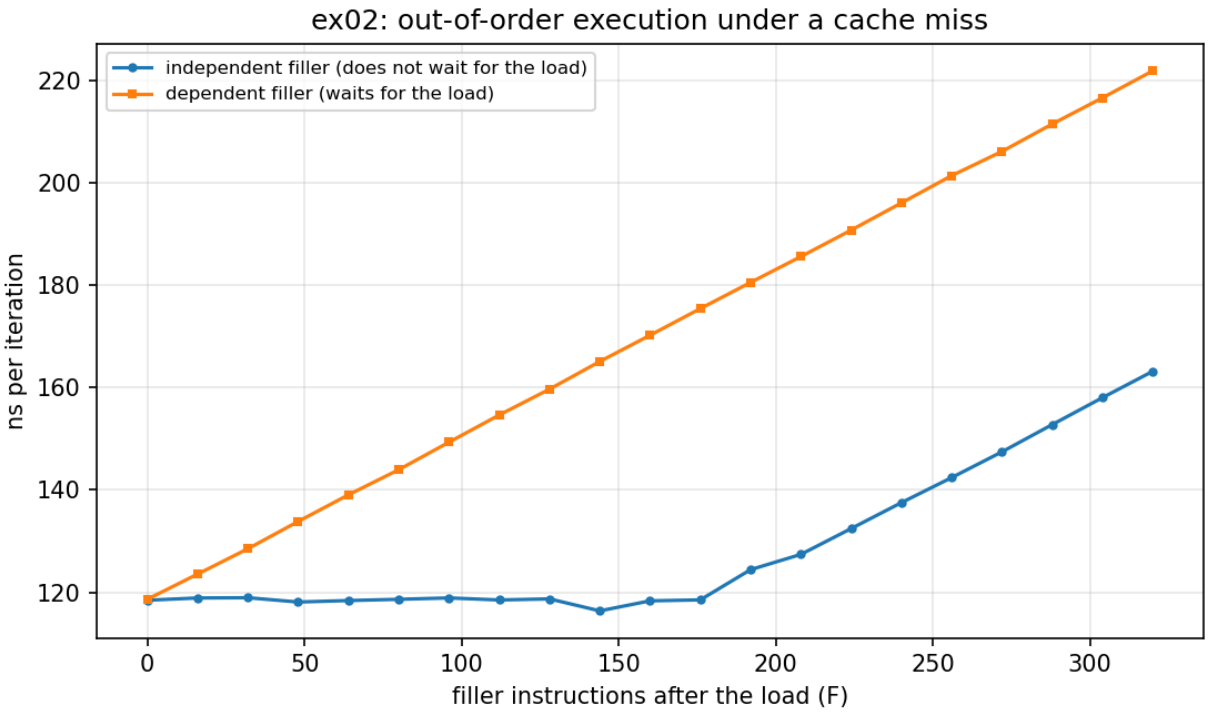


Figure 17: Elevation E-core OOO Execution Data

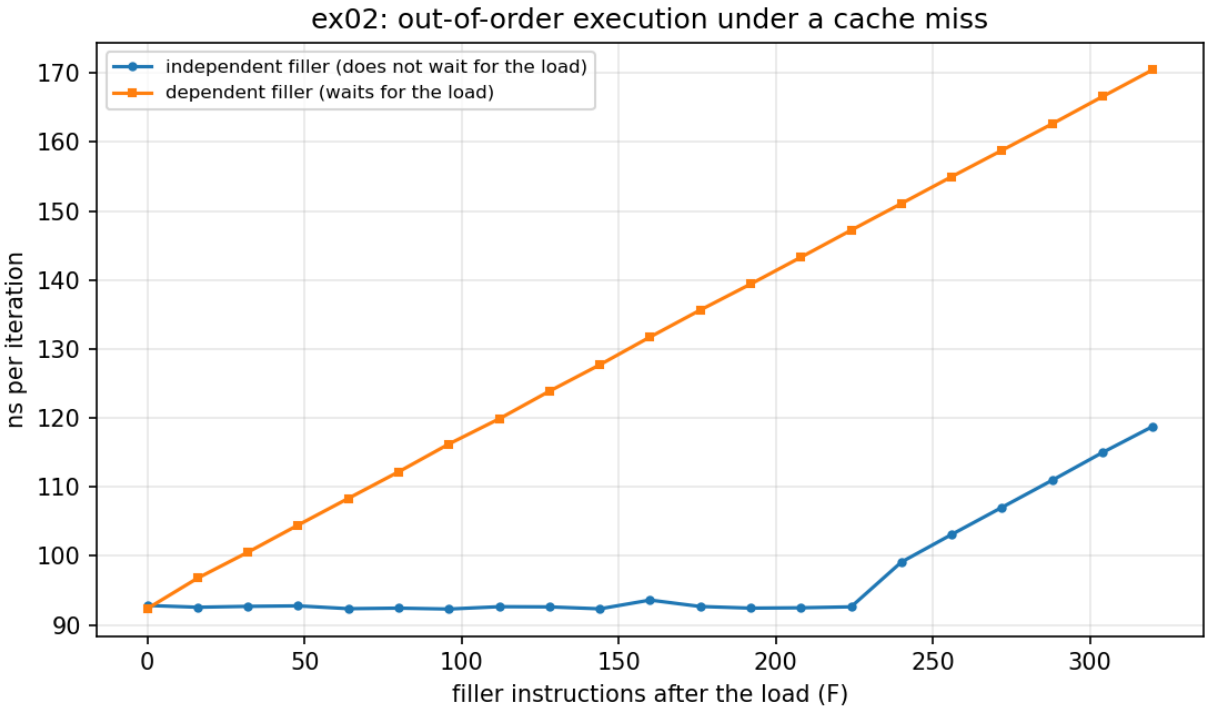


Figure 18: Elevation P-core OOO Execution Data

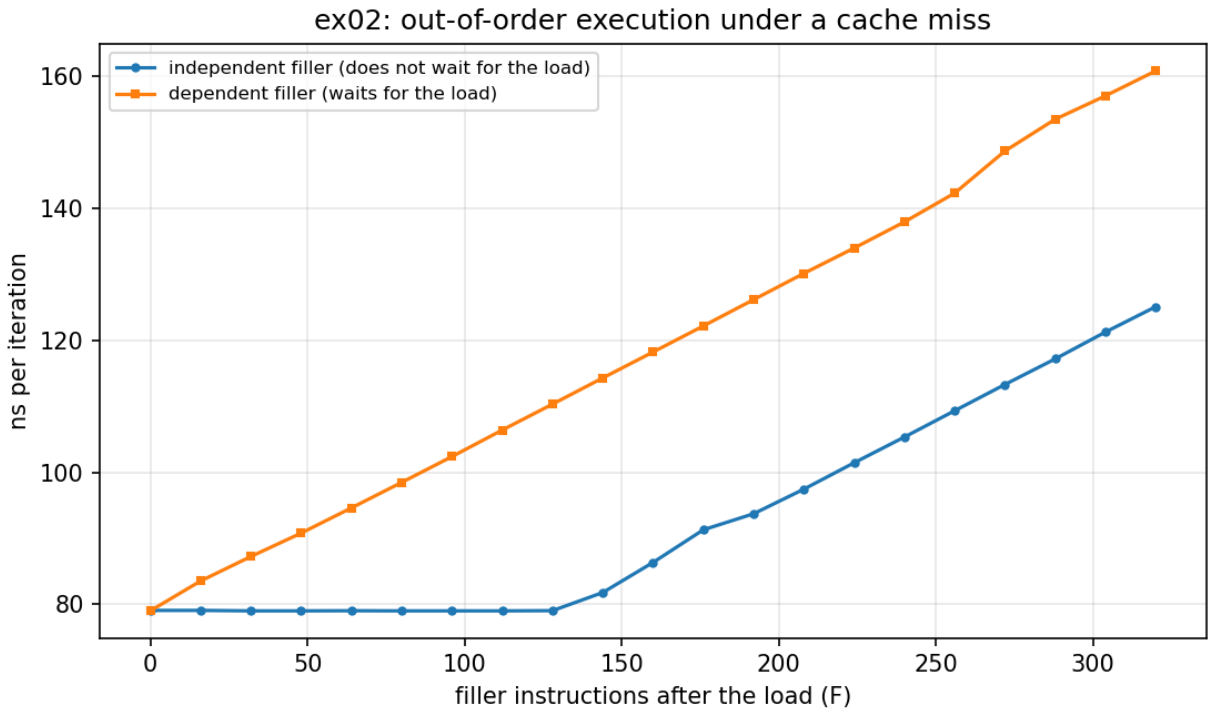


Figure 19: Ookay OOO Execution Data

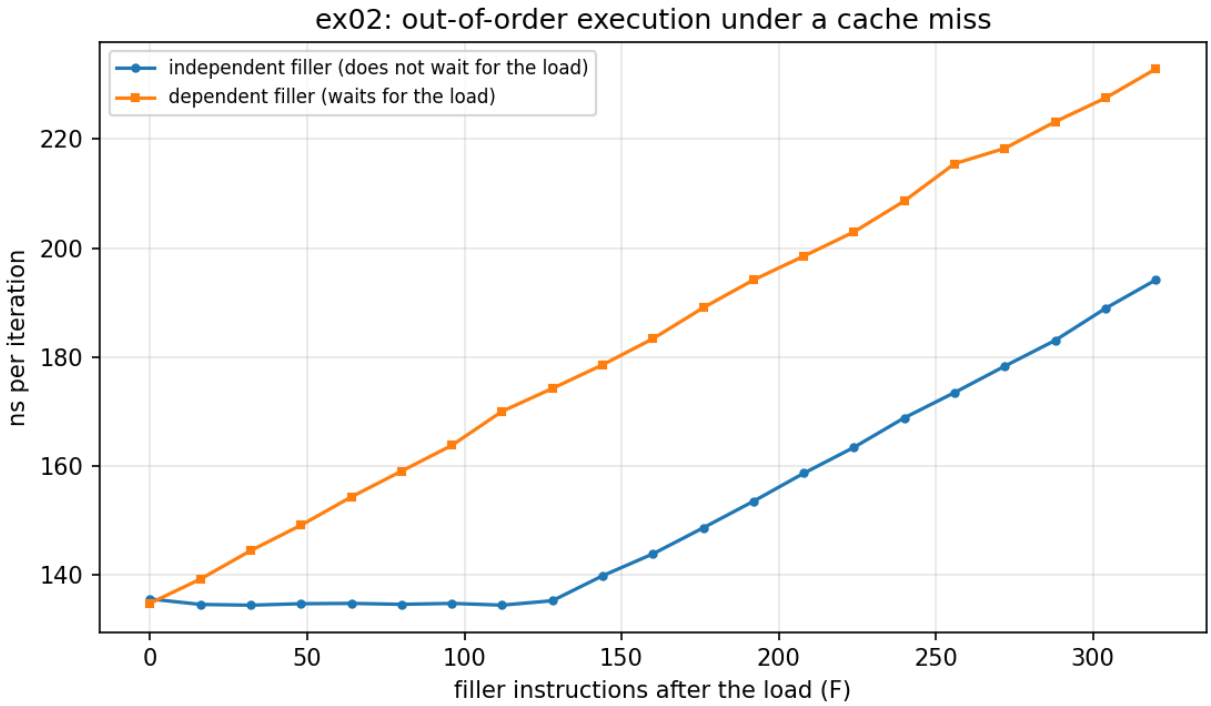


Figure 20: Skylark OOO Execution Data

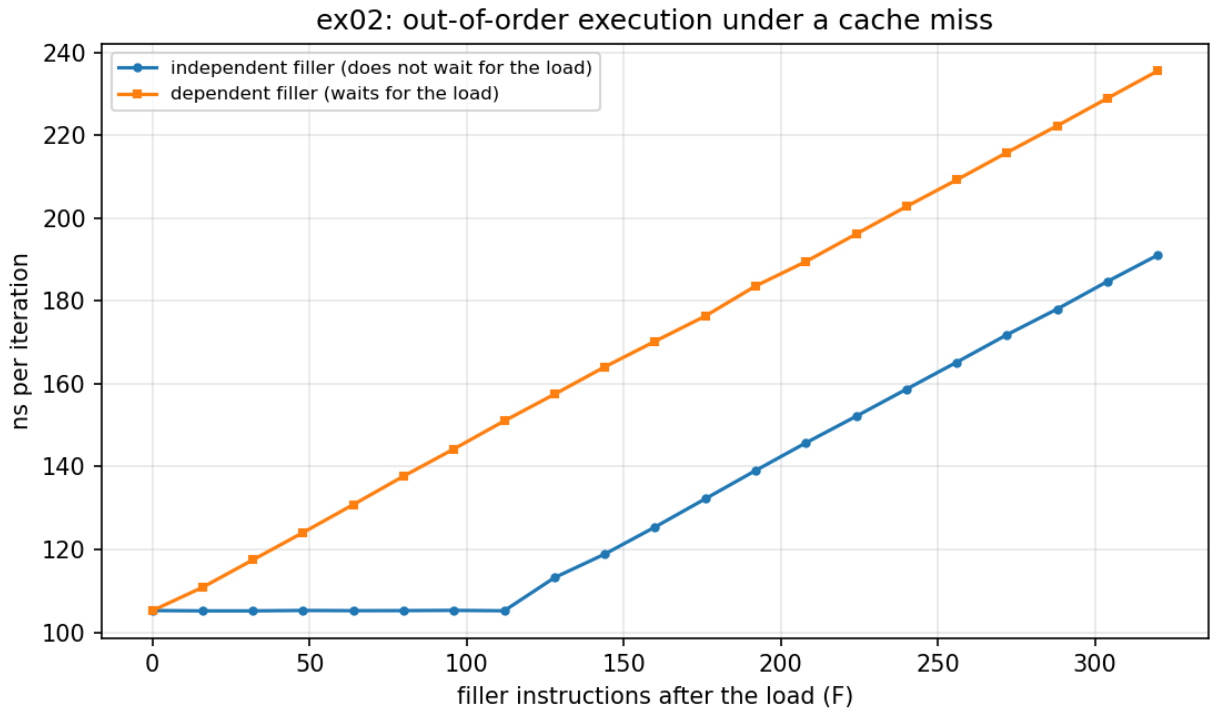


Figure 21: Sunbird OOO Execution Data

All machines were confirmed to use out-of-order execution. Analyzing the data, the most outstanding piece is the P-cores of the hybrid machines do not increase in time per filler instructions after the load until around 225 instructions whereas all the other machines start to increase in time before 200 instructions. This can be explained by the P-cores of the hybrid machines having a larger capacity to handle independent instructions before resources are exhausted compared to the other machines. The only difference in results between the SGX machine, Artemisia, and the other machines is the miss latency tends to be higher in Artemisia, but not by much. In conclusion, a claim can be made that the P-cores of the hybrid machines are more efficient in out-of-order execution, but there is minimal difference in the efficiency of SGX and non-SGX processors.

Below, Listing 3, is the main code snippet used to test the out-of-order execution of each machine. This can be found in the repository at `CSC591-Project3‣exercises‣ex02_ooo‣ooo_test.c`

Listing 4: OOO Code Block

```
// ooo_test.c - Is execution out of order? (x86-64, Linux, GCC/Clang)
//
// IDEA. A load that misses all caches takes ~100 ns (hundreds of cycles). Put F filler instructions right after it:
//   indep : filler 'add %r11,%rbx'  does NOT depend on the load  -> an out-of-order core executes it WHILE the load is
//                                   outstanding, so time per iteration stays ~ the miss latency (flat in F)
//   dep   : filler 'add %rax,%rbx'  depends on the loaded value  -> it cannot start before the load returns, so the F
//                                   adds (1 cycle each) are added AFTER the miss: time grows linearly in F
// An in-order core would show linear growth for BOTH. The two filler instructions are identical in size and cost; only
// the dependence differs. The next iteration's load address depends on BOTH this load's value and the end of the filler
// chain (all added values are 0, so the address is unchanged): the filler chain is on the critical path in both variants,
// and misses of different iterations cannot overlap. Independent filler therefore costs max(L, F cycles), dependent
// filler costs L + F cycles. The F fillers are entered through an indirect jump into a sled of identical
// 3-byte instructions (no JIT, same technique as ex04).
//
// OUTPUT (CSV on stdout): mode,f,rep,ns_per_iter

#include "bench_common.h"
#include <getopt.h>
#include <stdlib.h>
#include <string.h>
#include <sys/mman.h>
```

```
#if !defined(__x86_64__)
#error "ooo_test.c is x86-64 only"
#endif

#define XSTR(x) #x
#define STR(x) XSTR(x)
#define MAX_F 512
#define LINE 64UL
#define HUGE_SZ (2UL << 20)

// FILLER is 3 bytes (REX + 01 + modrm) in both variants, so the sled entry is 3*F bytes before its end.
#define KERNEL(name, FILLER_INSN)                                                                  \
    __attribute__((noinline)) static void name(const uint64_t *offs, const char *base, uint64_t iters,         \
                                               uint64_t nbytes) {                                  \
        __asm__ volatile("xor    %%eax, %%eax           \n\t"   /* rax = value of previous load (always 0)  */ \
                         "xor    %%ebx, %%ebx           \n\t"   /* rbx = end of the filler chain (stays 0)  */ \
                         "xor    %%r11d, %%r11d         \n\t"   /* r11 = constant 0, independent of the load */ \
                         "lea    3f(%%rip), %%r10       \n\t"                                                \
                         "sub    %[nb], %%r10           \n\t"   /* entry point: F fillers before label 3    */ \
                         "1:                            \n\t"                                                \
                         "mov    (%%rdi), %%rcx         \n\t"   /* offset for this iteration's load         */ \
                         "add    %%rax, %%rcx           \n\t"   /* address depends on previous load (adds 0) */ \
                         "add    %%rbx, %%rcx           \n\t"   /* ... and on the filler chain end (adds 0)  */ \
                         "mov    (%%rsi, %%rcx), %%rax  \n\t"   /* the load: DRAM miss                       */ \
                         "jmp    *%%r10                 \n\t"                                                \
                         ".rept " STR(MAX_F) "          \n\t" FILLER_INSN "\n\t" ".endr\n\t"                 \
                         "3:                            \n\t"                                                \
                         "add    $8, %%rdi              \n\t"                                                \
                         "dec    %%rdx                  \n\t"                                                \
                         "jnz    1b                     \n\t"                                                \
                         : "+D"(offs), "+d"(iters)                                                           \
                         : "S"(base), [nb] "r"(nbytes)                                                       \
                         : "rax", "rbx", "rcx", "r10", "r11", "cc", "memory");                               \
    }
KERNEL(k_indep, "add %%r11, %%rbx")
KERNEL(k_dep, "add %%rax, %%rbx")

static uint64_t rng = 88172645463325252ULL;

int main(int argc, char **argv) {
    int core = -1, reps = 9, s = 0, e = 320, t = 8, c;
    size_t mb = 512;
    uint64_t iters = 20000;
    while ((c = getopt(argc, argv, "c:m:i:r:s:e:t:h")) != -1) {
        switch (c) {
        case 'c': core = atoi(optarg); break;
        case 'm': mb = strtoul(optarg, 0, 10); break;
        case 'i': iters = strtoull(optarg, 0, 10); break;
        case 'r': reps = atoi(optarg); break;
        case 's': s = atoi(optarg); break;
        case 'e': e = atoi(optarg); break;
        case 't': t = atoi(optarg); break;
        default: fprintf(stderr, "usage: %s [-c core] [-m MiB] [-i iters] [-r reps] [-s start] [-e end] [-t step]\n", argv[0]); return 1;
        }
    }
    if (e > MAX_F || t < 1) { fprintf(stderr, "max F is %d\n", MAX_F); return 1; }
    if (core >= 0 && pin_to_cpu(core)) { perror("sched_setaffinity"); return 1; }

    size_t size = mb << 20;
    char *raw = mmap(NULL, size + HUGE_SZ, PROT_READ | PROT_WRITE, MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
    if (raw == MAP_FAILED) { perror("mmap"); return 1; }
    char *buf = (char *)(((uintptr_t)raw + HUGE_SZ - 1) & ~(HUGE_SZ - 1));
    madvise(buf, size, MADV_NOHUGEPAGE);
    for (size_t i = 0; i < size; i += 4096) ((volatile char *)buf)[i] = 0;   // private zero pages (the loaded value must be 0)
    uint64_t *offs = malloc(iters * sizeof(uint64_t));
    uint64_t nlines = size / LINE;

    printf("# arch=%s core=%d buf_MiB=%zu iters=%lu reps=%d core_ghz=%.4f\n", BENCH_ARCH, core, mb, (unsigned long)iters, reps, core_ghz());
    printf("mode,f,rep,ns_per_iter\n");
    for (int r = -1; r < reps; r++) {                   // rep-major; r=-1 is a warm-up pass
        for (int f = s; f <= e; f += t) {
            for (int m = 0; m < 2; m++) {
                for (uint64_t i = 0; i < iters; i++) offs[i] = (xorshift64(&rng) % nlines) * LINE;
                for (uint64_t i = 0; i < iters; i++) clflush_line(buf + offs[i]);
                mfence();
                double t0 = now_ns();
                (m == 0 ? k_indep : k_dep)(offs, buf, iters, 3UL * (uint64_t)f);
                double t1 = now_ns();
                if (r >= 0) printf("%s,%d,%d,%.2f\n", m == 0 ? "indep" : "dep", f, r, (t1 - t0) / iters);
            }
        }
        fflush(stdout);
    }
    return 0;
}
```

7 Branch Prediction

The fourth experiment conducted was verifying if a processor utilized branch predictions. Each experiment was ran three times. At the end of the three test runs, data was collected and summarized into a table. The CSV summary table, Table 6, can be found in the repository at `CSC591-Project3‣report‣tables‣ex01_branch_prediction_summary`. The figures shown below of the each of the machine's data is representation of the last test ran to determine if the processor uses branch predictions. Each of the machines respective figures can be found in the repository at `CSC591-Project3‣data‣<machine_name>‣ex03_branch_prediction‣<run_number>‣plot.png`

Machine	Core	R^2 Value	Penalty Cycles	Penalty Time (ns)	Time for Random 50% (ns)
artemisia	cpu29	0.972	24.22	9.02	5.087
charnwood	cpu2	0.987	19.1	6.49	3.57
crux	cpu3	0.986	14.97	5.25	2.901
descent	ecore	0.999	25.6	6.96	3.974
descent	pcore	0.994	25.53	5.26	2.79
elevation	ecore	0.999	27.0	8.23	4.692
elevation	pcore	0.994	26.29	6.04	3.194
ookay	cpu2	0.987	16.4	5.9	3.237
skylark	cpu2	0.989	25.21	7.68	4.244
sunbird	cpu8	0.98	20.01	8.7	4.921

Table 6: Processor Branch Prediction Summary

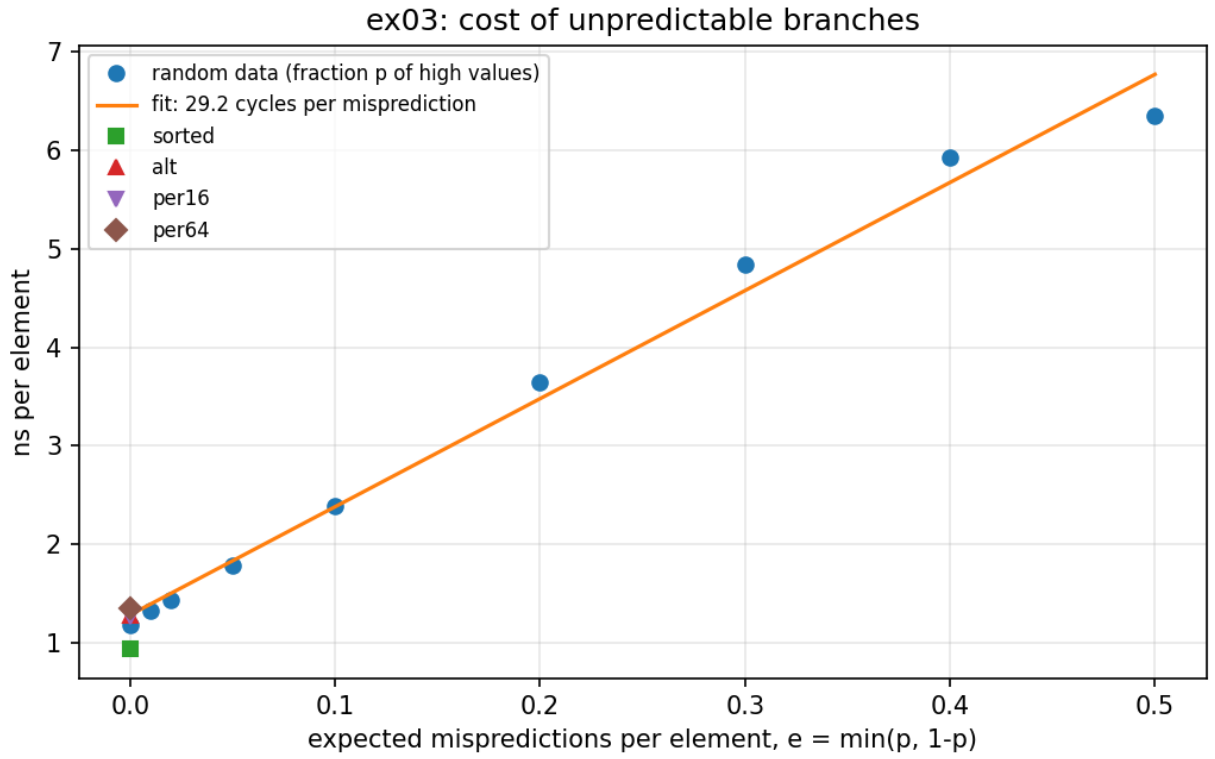


Figure 22: Artemisia Branch Prediction Data

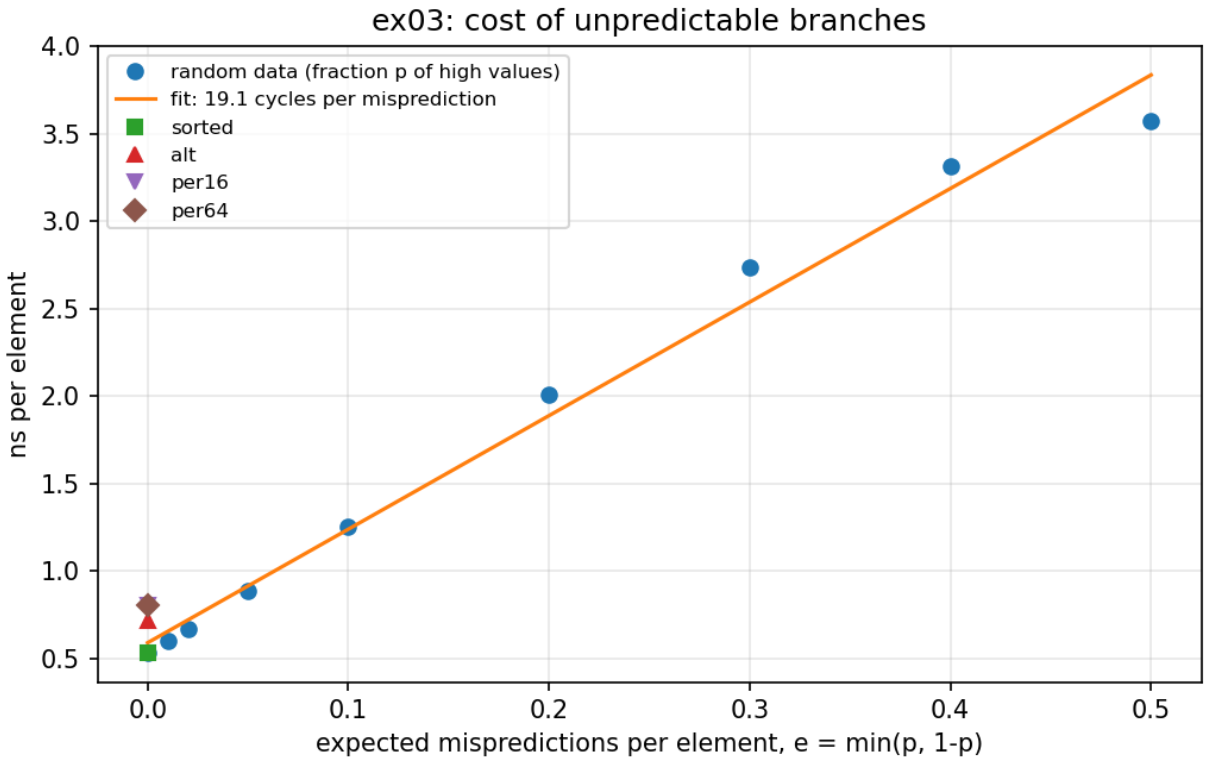


Figure 23: Charnwood Branch Prediction Data

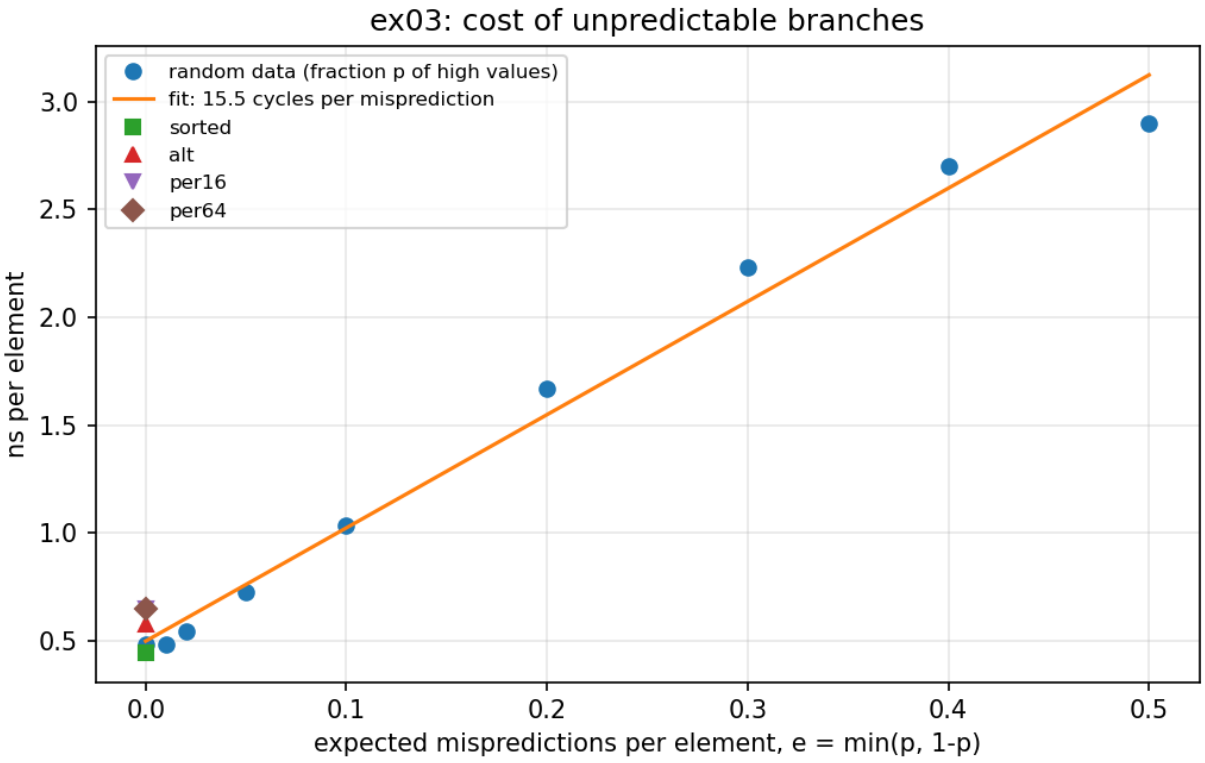


Figure 24: Crux Branch Prediction Data

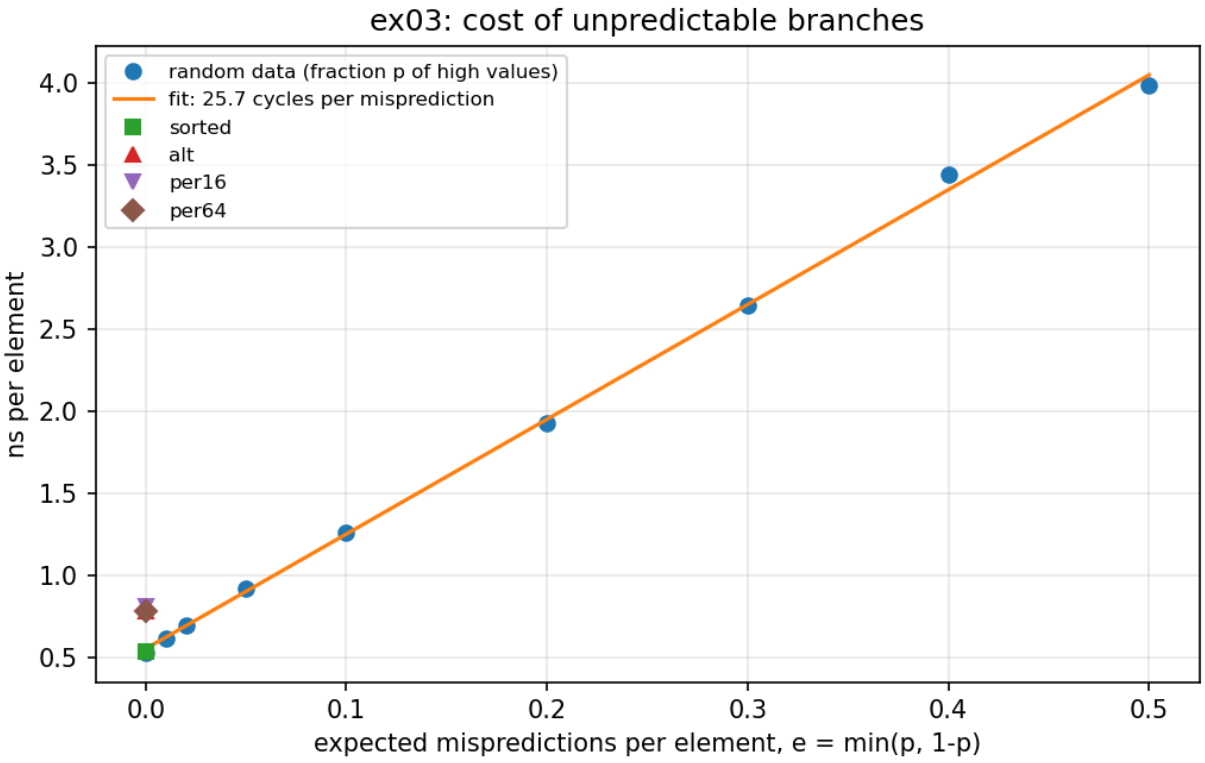


Figure 25: Descent E-core Branch Prediction Data

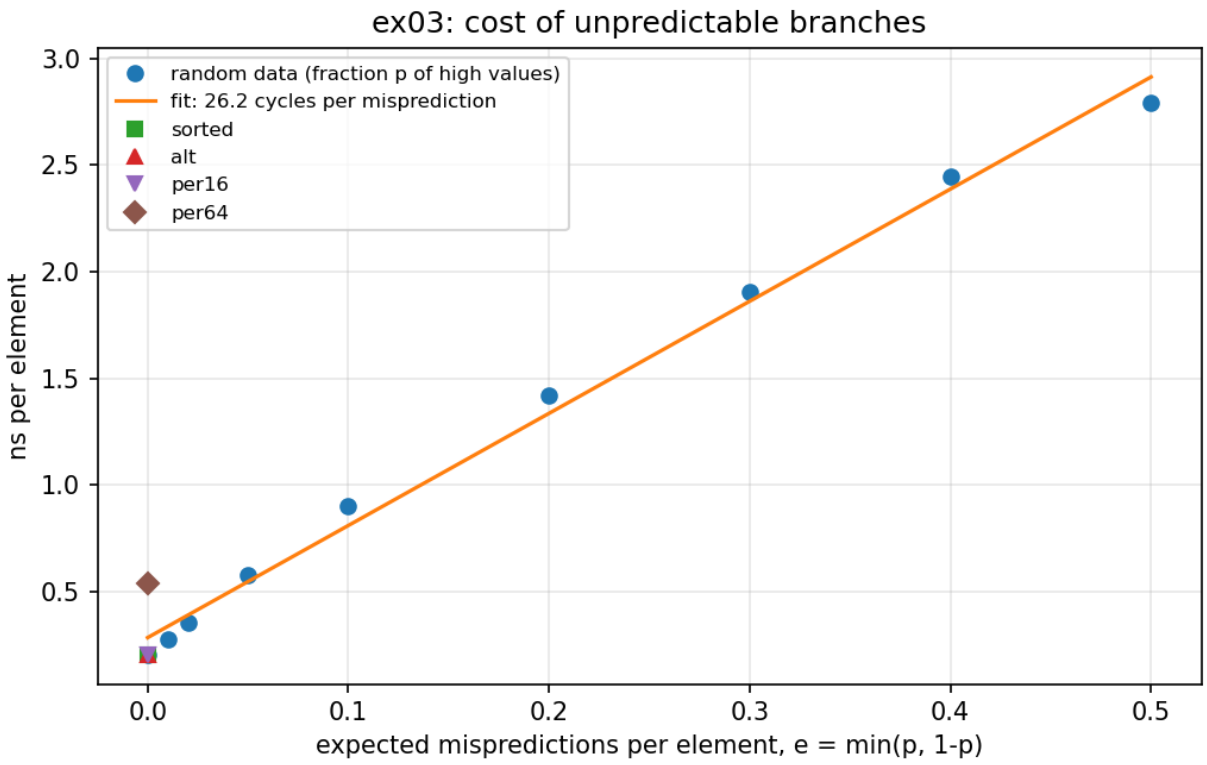


Figure 26: Descent P-core Branch Prediction Data

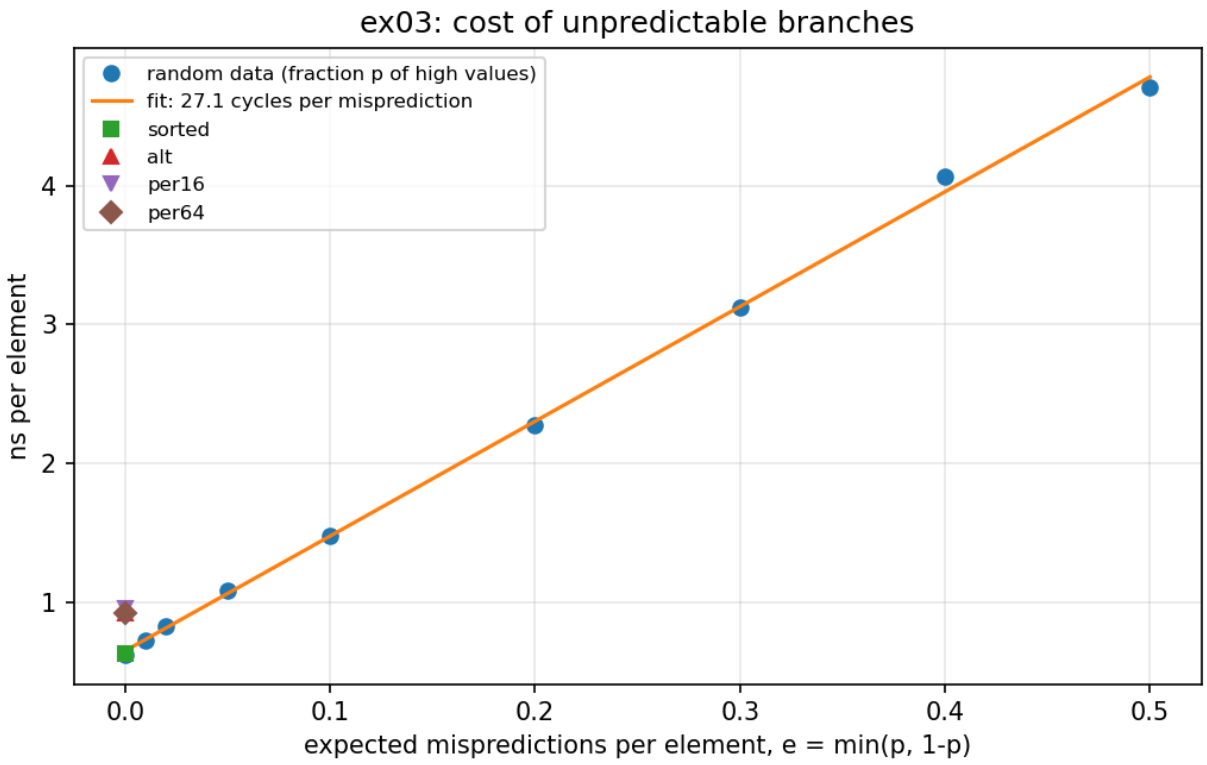


Figure 27: Elevation E-core Branch Prediction Data

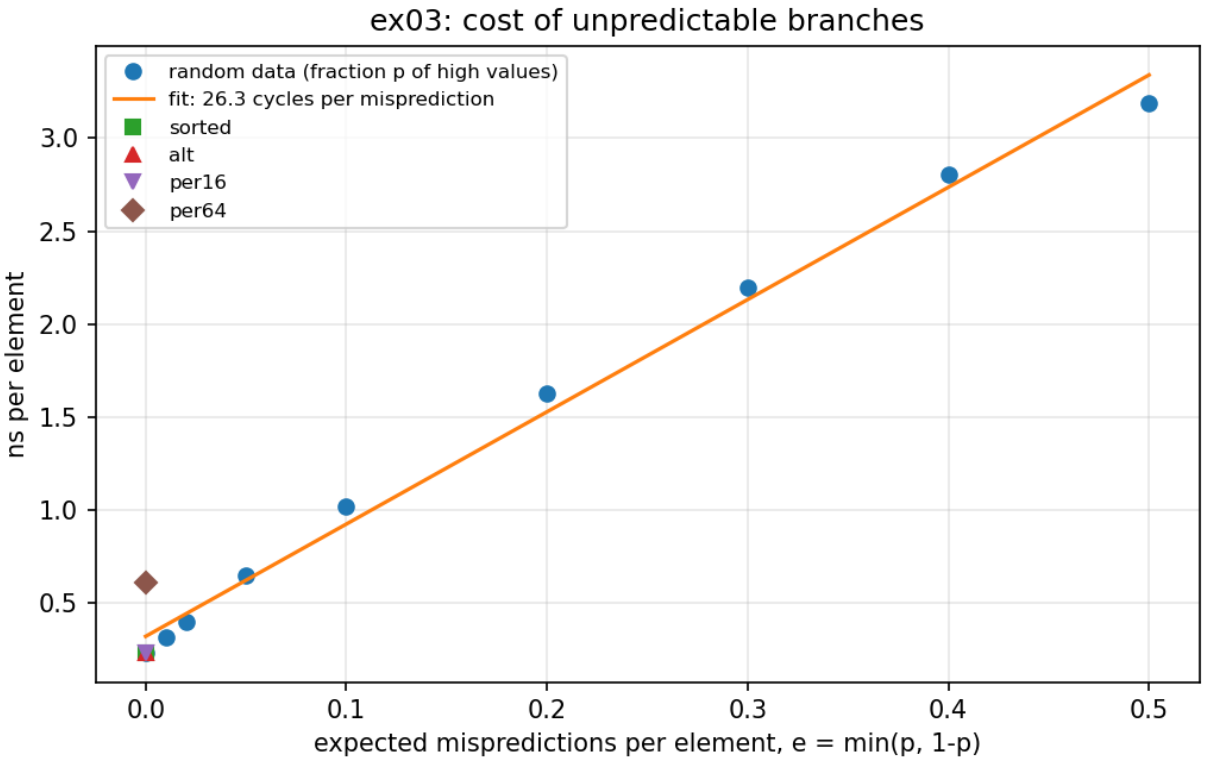


Figure 28: Elevation P-core Branch Prediction Data

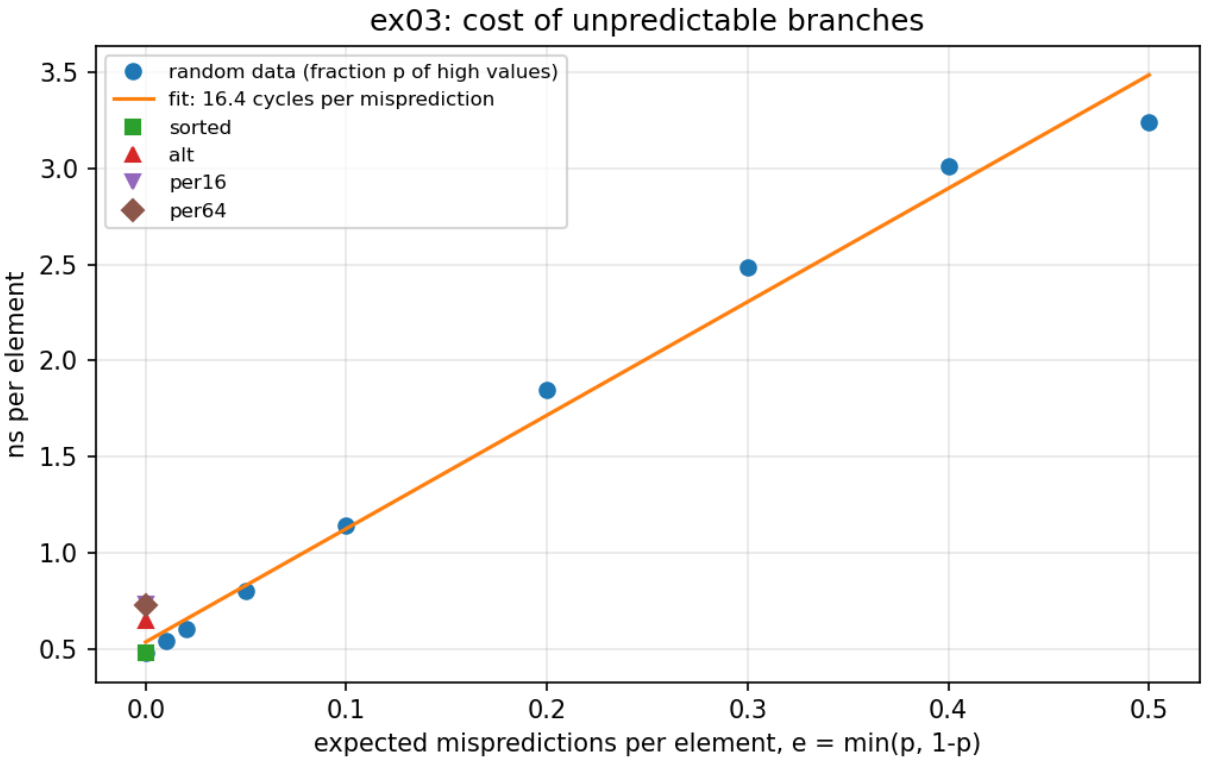


Figure 29: Ookay Branch Prediction Data

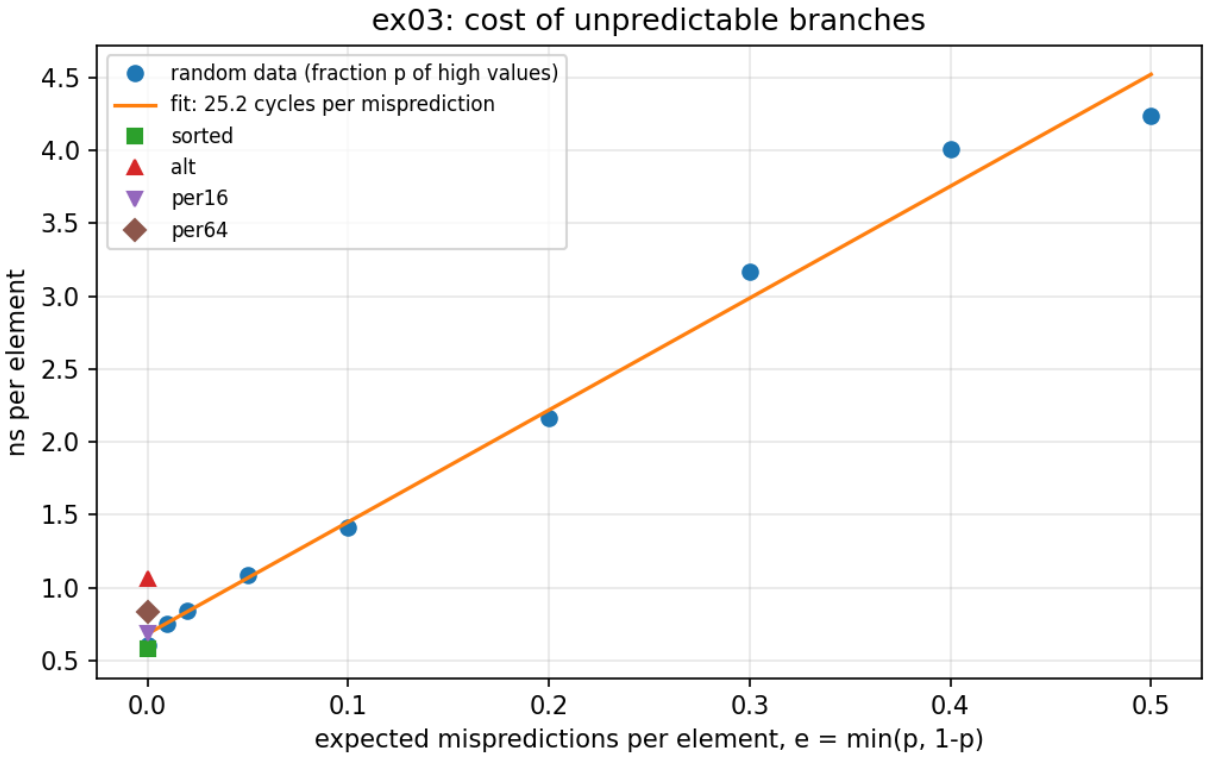


Figure 30: Skylark Branch Prediction Data

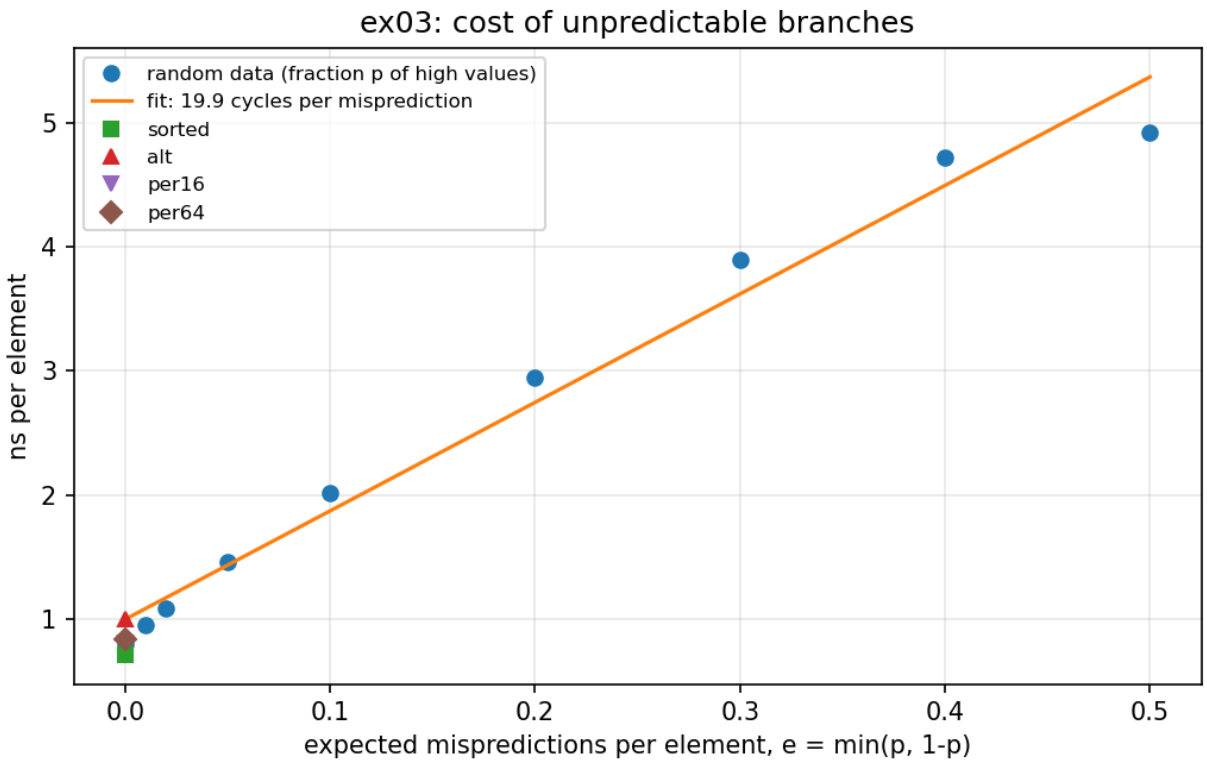


Figure 31: Sunbird Branch Prediction Execution Data

All of the processors were confirmed to use branch prediction. Analyzing the data, the e-cores of the hybrid machine had the highest R^2 value at 0.999. However, the p-cores had the highest value of penalty cycles. The SGX enabled machine, Artemisia, had the highest penalty time and also had the highest time for random 50%. The shortest times for random 50% were Descent p-core and Crux, both with values under three. A predicted claim can be made that SGX and p-cores of hybrid machines have higher miss penalties and also take longer to execute a random workload, making non-SGX and non-hybrid processors more efficient for random, unpredictable workloads.

Below, Listing 4, is the main code snippet used to test the branch prediction of each machine. This can be found in the repository at `CSC591-Project3‣exercises‣ex02_branch_prediction‣branch_test.c`

Listing 5: Branch Prediction Code Block

```
// branch_test.c - Does the processor use branch prediction? How much does a misprediction cost? (x86-64, Linux)
//
// IDEA. One data-dependent conditional branch per array element, written in inline asm so the compiler cannot turn it
// into a conditional move:      if (data[i] >= 128) sum += data[i];
// The work per element is the same for every data set; only the PREDICTABILITY of the branch differs:
//   pNN     i.i.d. random data, a fraction NN% of elements >= 128. Best possible predictor mispredicts min(p,1-p) of them,
//           so time per element should grow LINEARLY with that fraction; the slope is the cost of one misprediction.
//   sorted  same data as p50 but sorted: one transition in the whole array -> ~0 mispredictions
//   alt     50,200,50,200...              period 2
//   per16   a random 16-element pattern repeated    period 16
//   per64   a random 64-element pattern repeated    period 64
// A history-based predictor learns the periodic patterns (fast, like 'sorted'); random data cannot be learned.
// The array is 4 MiB and each pass uses fresh random data, so the predictor cannot memorize it.
//
// OUTPUT (CSV on stdout): mode,e,rep,ns_per_elem     (e = expected mispredictions per element for an ideal predictor,
//                                                      -1 for the periodic/sorted modes: not used in the regression)

#include "bench_common.h"
#include <getopt.h>
#include <stdlib.h>
#include <string.h>
```

```
#if !defined(__x86_64__)
#error "branch_test.c is x86-64 only"
#endif

static uint64_t rng = 88172645463325252ULL;
static volatile uint64_t sink;

// the measured kernel: the branch is `jb` on the compare
__attribute__((noinline)) static uint64_t kernel(const uint8_t *d, uint64_t n) {
    uint64_t sum;
    __asm__ volatile("xor   %%eax, %%eax        \n\t"
                     "1:                        \n\t"
                     "movzbl (%[d]), %%ecx      \n\t"
                     "cmp    $128, %%ecx        \n\t"
                     "jb     2f                 \n\t"   // data-dependent branch (taken when value < 128)
                     "add    %%rcx, %%rax       \n\t"   // only executed when value >= 128
                     "2:                        \n\t"
                     "inc    %[d]               \n\t"
                     "dec    %[n]               \n\t"
                     "jnz    1b                 \n\t"
                     : "=&a"(sum), [d] "+r"(d), [n] "+r"(n)
                     :
                     : "rcx", "cc", "memory");
    return sum;
}

struct mode { const char *name; int pct; double e; };   // pct >= 0: i.i.d. with pct% high values; pct < 0: special
static const struct mode modes[] = {
    {"p00", 0, 0.00}, {"p01", 1, 0.01}, {"p02", 2, 0.02}, {"p05", 5, 0.05}, {"p10", 10, 0.10}, {"p20", 20, 0.20},
    {"p30", 30, 0.30}, {"p40", 40, 0.40}, {"p50", 50, 0.50},
    {"sorted", -1, -1}, {"alt", -2, -1}, {"per16", -3, -1}, {"per64", -4, -1},
};
#define NM (int)(sizeof(modes) / sizeof(modes[0]))

static void fill(uint8_t *d, uint64_t n, int pct) {
    if (pct >= 0 || pct == -1) {                        // i.i.d. (or sorted version of p50)
        int p = pct >= 0 ? pct : 50;
        uint64_t thr = (uint64_t)((double)p / 100.0 * 4294967296.0), hi = 0;
        for (uint64_t i = 0; i < n; i++) {
            int high = (xorshift64(&rng) & 0xFFFFFFFFULL) < thr;
            d[i] = high ? 200 : 50;
            hi += high;
        }
        if (pct == -1) { memset(d, 50, n - hi); memset(d + (n - hi), 200, hi); }   // sorted
    } else if (pct == -2) {
        for (uint64_t i = 0; i < n; i++) d[i] = (i & 1) ? 200 : 50;
    } else {
        int period = pct == -3 ? 16 : 64;
        uint8_t pat[64];
        for (int i = 0; i < period; i++) pat[i] = (xorshift64(&rng) & 1) ? 200 : 50;
        for (uint64_t i = 0; i < n; i++) d[i] = pat[i % period];
    }
}

int main(int argc, char **argv) {
    int core = -1, reps = 9, c;
    uint64_t n = 1UL << 22;
    while ((c = getopt(argc, argv, "c:r:n:h")) != -1) {
        if (c == 'c') core = atoi(optarg);
        else if (c == 'r') reps = atoi(optarg);
        else if (c == 'n') n = strtoull(optarg, 0, 10);
        else { fprintf(stderr, "usage: %s [-c core] [-r reps] [-n elements]\n", argv[0]); return 1; }
    }
    if (core >= 0 && pin_to_cpu(core)) { perror("sched_setaffinity"); return 1; }
    uint8_t *d = malloc(n);
    if (!d) { perror("malloc"); return 1; }

    printf("# arch=%s core=%d elements=%lu reps=%d core_ghz=%.4f\n", BENCH_ARCH, core, (unsigned long)n, reps, core_ghz());
    printf("mode,e,rep,ns_per_elem\n");
    for (int r = -1; r < reps; r++) {                   // rep-major, r=-1 warm-up
        for (int m = 0; m < NM; m++) {
            fill(d, n, modes[m].pct);
            double t0 = now_ns();
            sink = kernel(d, n);
            double t1 = now_ns();
            if (r >= 0) printf("%s,%.2f,%d,%.4f\n", modes[m].name, modes[m].e, r, (t1 - t0) / (double)n);
        }
        fflush(stdout);
    }
    return 0;
}
```

8 Pipeline Depth

The fifth experiment conducted was analyzing the processor depth of each machine using data gathered from branch predictions. Each experiment was ran three times. At the end of the three test runs, data was collected and summarized into a table. The CSV summary table, Table 7, can be found in the repository at `CSC591-Project3‣report‣tables‣ex15_pipeline_depth`. The pipeline depth graph, Figure 32, can be found in the repository at `CSC591-Project3‣reports‣figures‣ex15_pipeline_depth.png`

Machine	Core	Median Penalty Cycles	Median Two Point Cycles
artemisia	cpu29	24.2	24.3
charnwood	cpu2	19.1	17.9
crux	cpu3	15.0	14.0
descent	ecore	25.6	25.3
descent	pcore	25.5	25.1
elevation	ecore	27.0	26.7
elevation	pcore	26.3	25.9
ookay	cpu2	16.4	15.3
skylark	cpu2	25.2	24.0
sunbird	cpu8	20.0	19.4

Table 7: Processor Pipeline Depth Summary

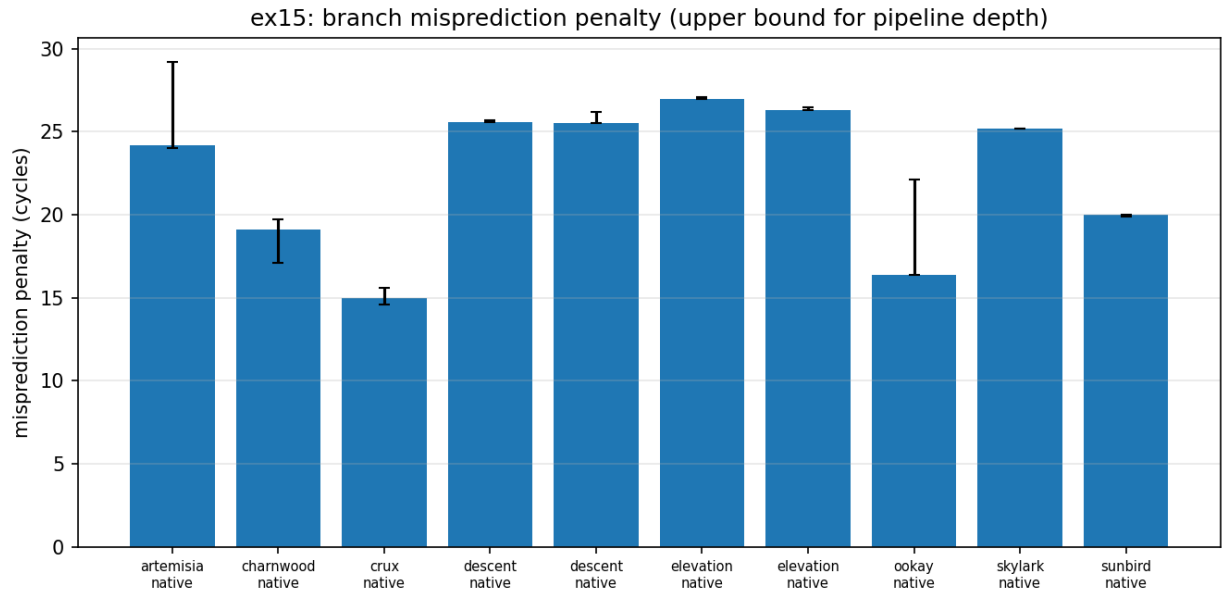


Figure 32: Pipeline Depth Summary

Similar to the branch prediction data, Artemisia and the hybrid machines had higher penalty cycles as well as a higher median recovery cycle value as compared to the other machines. The

non-SGX and non-hybrid processors produced better and more efficient results than the SGX and hybrid machines. A claim can be made that due to the high miss penalties of the SGX and hybrid processors, the non-SGX and non-hybrid machines have a shallower pipeline depth, making their unpredicted instruction execution time more efficient.

Below, Listing 5, is the main code snippet used to test the pipeline depth of each machine. This can be found in the repository at `CSC591-Project3 ▸ exercises ▸ ex15_pipeline_depth ▸ analyze_ex15.py`

Listing 6: Pipeline Depth Code Block

```
#!/usr/bin/env python3
"""ex15: pipeline depth from the branch misprediction penalty measured in ex03 (native and, if present, enclave).

usage (repo root):  python3 exercises/ex15_pipeline_depth/analyze_ex15.py
No new measurement: it reads data/<machine>/ex03_branch_prediction[_sgx]/<run>/summary.txt (KEY=value lines).
penalty (cycles) = ns lost per misprediction x core clock. Both estimates from ex03 are used:
  PENALTY_CYCLES              slope of time vs fraction of unpredictable branches (random data, 9 fractions)
  PENALTY_CYCLES_TWO_POINT    (random 50% - sorted) / 0.5
Writes report/tables/ex15_pipeline_depth.csv/.md and report/figures/ex15_pipeline_depth.png.
What the number means: it is the time from fetching a branch to resteering the front end after the branch resolves, i.e. the
front-end pipeline length PLUS the latency of the data the branch waits for (here an L1 load and a compare). So it is an UPPER
bound on the front-end depth; discuss that gap in the report.
"""
import csv, os, re, statistics as st, sys
from collections import defaultdict

root = os.path.dirname(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
data = os.path.join(root, "data")
read = lambda p: open(p, errors="replace").read() if os.path.exists(p) else ""
kvs = lambda t: dict(m.groups() for m in re.finditer(r"^([A-Z][A-Z0-9_]*)=(.*)$", t, re.M))

G = defaultdict(list)
for machine in sorted(os.listdir(data)) if os.path.isdir(data) else []:
    for ex in ("ex03_branch_prediction", "ex03_branch_prediction_sgx"):
        exdir = os.path.join(data, machine, ex)
        if not os.path.isdir(exdir):
            continue
        for run in sorted(os.listdir(exdir)):
            rd = os.path.join(exdir, run)
            if run.endswith("_latest") or os.path.islink(rd) or not os.path.isdir(rd):
                continue
            kv = kvs(read(os.path.join(rd, "summary.txt")))
            if kv.get("STATUS") != "ok":
                continue
            cfg = dict(x.split("=", 1) for x in read(os.path.join(rd, "run_config.txt")).split() if "=" in x)
            m = re.search(r"model name\s*:\s*(.*)", read(os.path.join(rd, "sysinfo", "cpu_model.txt")))
            G[(machine, "enclave" if ex.endswith("_sgx") else "native", cfg.get("label", "?"))].append(
                (float(kv["PENALTY_CYCLES"]), float(kv["PENALTY_CYCLES_TWO_POINT"]), float(kv["CORE_GHZ"]), m.group(1).strip() if m else "?"))
if not G:
    sys.exit("no confirmed ex03 runs found: run  bash scripts/fleet.sh all ex03_branch_prediction  first")

rows = []
for (machine, env, label), v in sorted(G.items()):
    pen, two, ghz = [x[0] for x in v], [x[1] for x in v], [x[2] for x in v]
    rows.append({"machine": machine, "cpu_model": v[0][3], "environment": env, "core_label": label, "runs": len(v),
                 "penalty_cycles_median": round(st.median(pen), 1), "penalty_min": round(min(pen), 1), "penalty_max": round(max(pen), 1),
                 "two_point_cycles_median": round(st.median(two), 1), "core_ghz": round(st.median(ghz), 2)})
os.makedirs(os.path.join(root, "report", "tables"), exist_ok=True)
os.makedirs(os.path.join(root, "report", "figures"), exist_ok=True)
base = os.path.join(root, "report", "tables", "ex15_pipeline_depth")
with open(base + ".csv", "w", newline="") as f:
    w = csv.DictWriter(f, fieldnames=list(rows[0])); w.writeheader(); w.writerows(rows)
with open(base + ".md", "w") as f:
    c = list(rows[0]); f.write("| " + " | ".join(c) + " |\n| " + "---| " * len(c) + "\n")
    for r in rows: f.write("| " + " | ".join(str(r[k]) for k in c) + " |\n")
print(read(base + ".md"))
print(f"wrote {base}.csv / .md")
if any(r["runs"] < 3 for r in rows):
    print("NOTE: some configurations have fewer than 3 confirmed ex03 runs")
try:
    import matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt
except ImportError:
    sys.exit("matplotlib not installed: table written, plot skipped")
fig, ax = plt.subplots(figsize=(9, 4.5))
lab = [f"{r['machine']}\n{r['environment']}" for r in rows]
ax.bar(range(len(rows)), [r["penalty_cycles_median"] for r in rows],
       yerr=[[r["penalty_cycles_median"] - r["penalty_min"] for r in rows], [r["penalty_max"] - r["penalty_cycles_median"] for r in rows]],
       capsize=3)
ax.set_xticks(range(len(rows))); ax.set_xticklabels(lab, fontsize=7)
ax.set_ylabel("misprediction penalty (cycles)"); ax.set_title("ex15: branch misprediction penalty (upper bound for pipeline depth)")
ax.grid(alpha=0.3, axis="y"); fig.tight_layout()
out = os.path.join(root, "report", "figures", "ex15_pipeline_depth.png"); fig.savefig(out, dpi=150); print("wrote", out)
```

9 TLB Miss Cost

The sixth experiment conducted was analyzing the TLB Miss Cost of different processors. Each experiment was ran three times. At the end of the three test runs, data was collected and summarized into a table. The CSV summary table, Table 7, can be found in the repository at `CSC591-Project3 ▸ report ▸ tables ▸ ex08_tlb_miss_cost_summary`. The figures shown below of the each of the machine's data is representation of the last test ran to analyze the TLB miss cost. Each of the machines respective figures can be found in the repository at `CSC591-Project3 ▸ data ▸ <machine_name> ▸ ex08_tlb_miss_cost ▸ <run_number> ▸ plot.png`

Machine	Core	L1 Miss Cycles	TLB Walk Cold Cycles	TLB Walk Cycles
artemisia	cpu29	7.0	58.6	22.9
charnwood	cpu2	9.4	219.1	38.9
crux	cpu3	8.6	243.3	29.6
descent	ecore	9.0	93.0	33.1
descent	pcore	7.0	53.2	47.8
elevation	ecore	9.0	236.6	33.6
elevation	pcore	7.0	51.3	29.6
ookay	cpu2	9.1	198.0	38.5
skylark	cpu2	7.2	55.8	33.8
sunbird	cpu8	8.0	47.6	28.9

Table 8: Processor TLB Miss Cost Summary

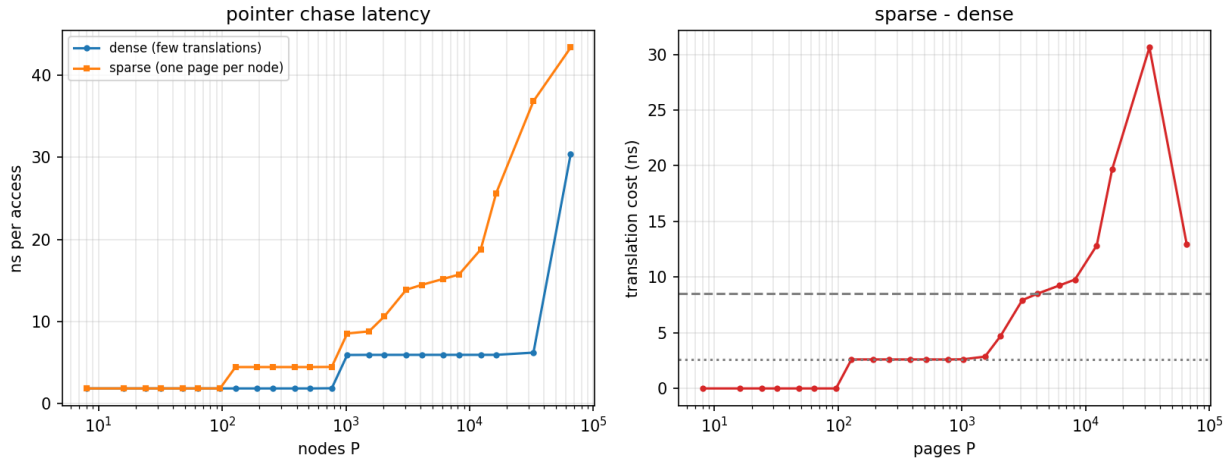


Figure 33: Artemisia TLB Miss Cost Data

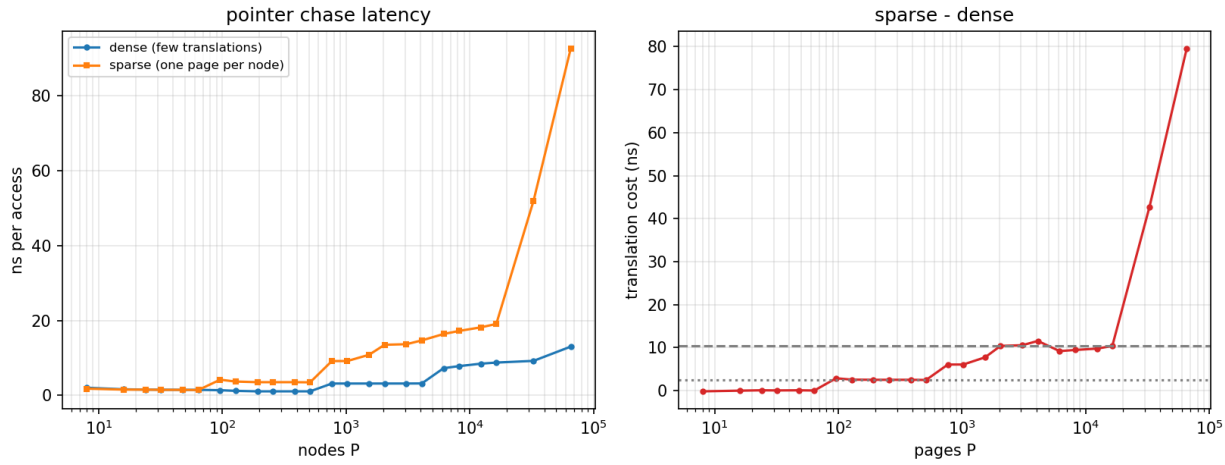


Figure 34: Charnwood TLB Miss Cost Data

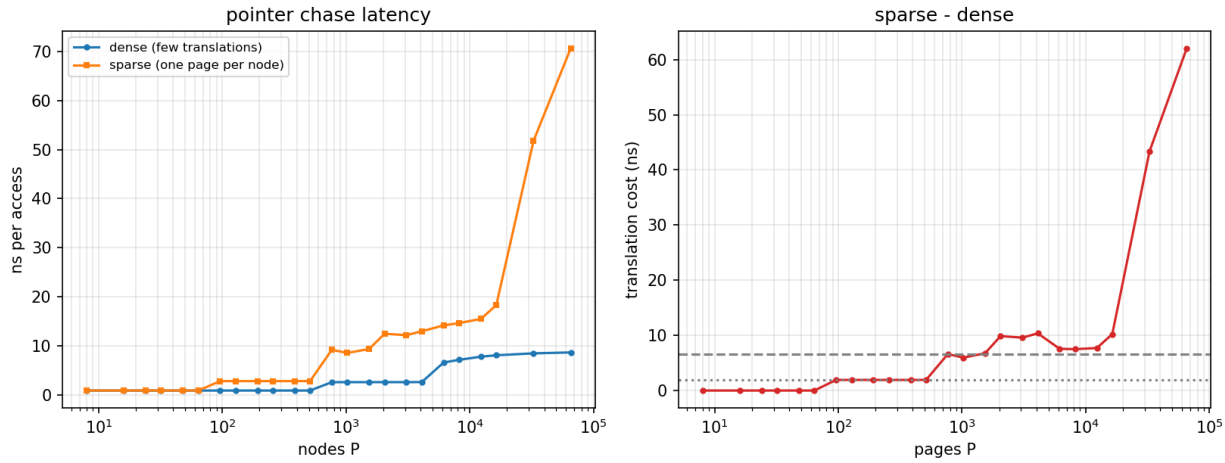


Figure 35: Crux TLB Miss Cost Data

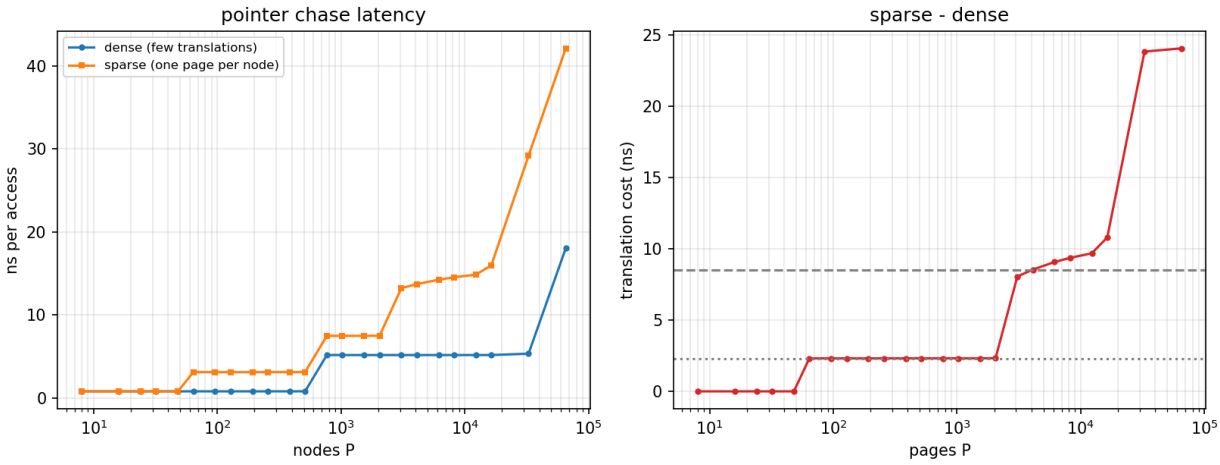


Figure 36: Descent E-core TLB Miss Cost Data

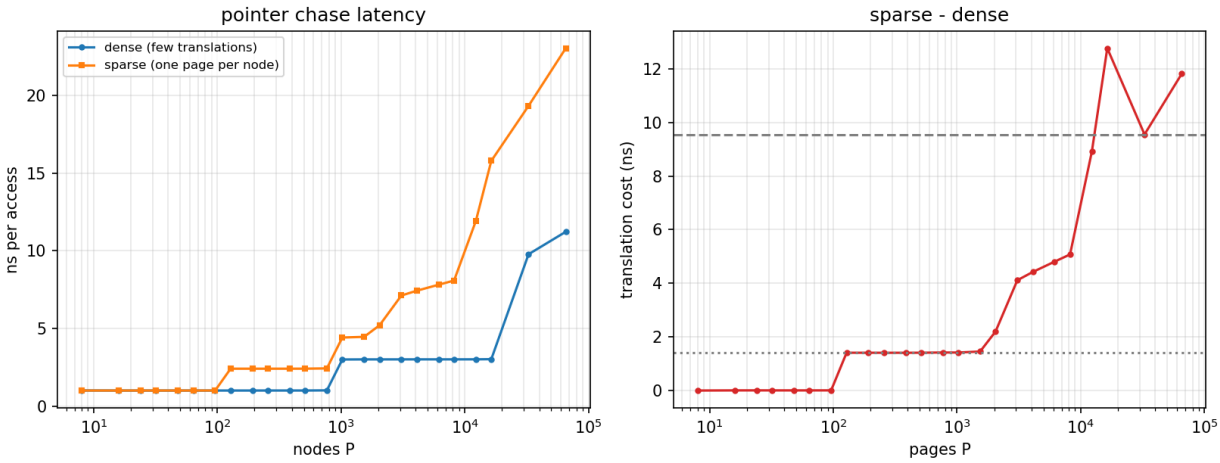


Figure 37: Descent P-core TLB Miss Cost Data

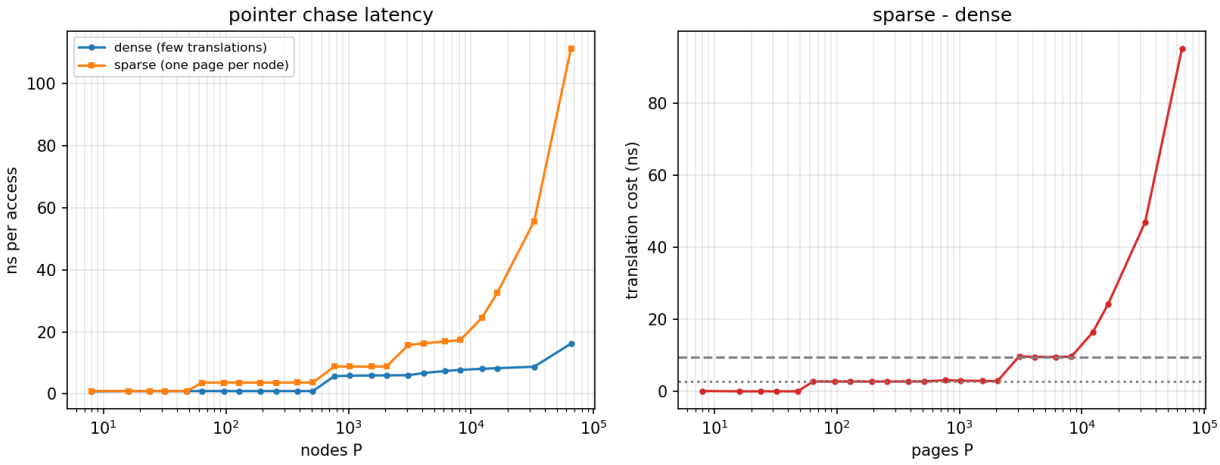


Figure 38: Elevation E-core TLB Miss Cost Data

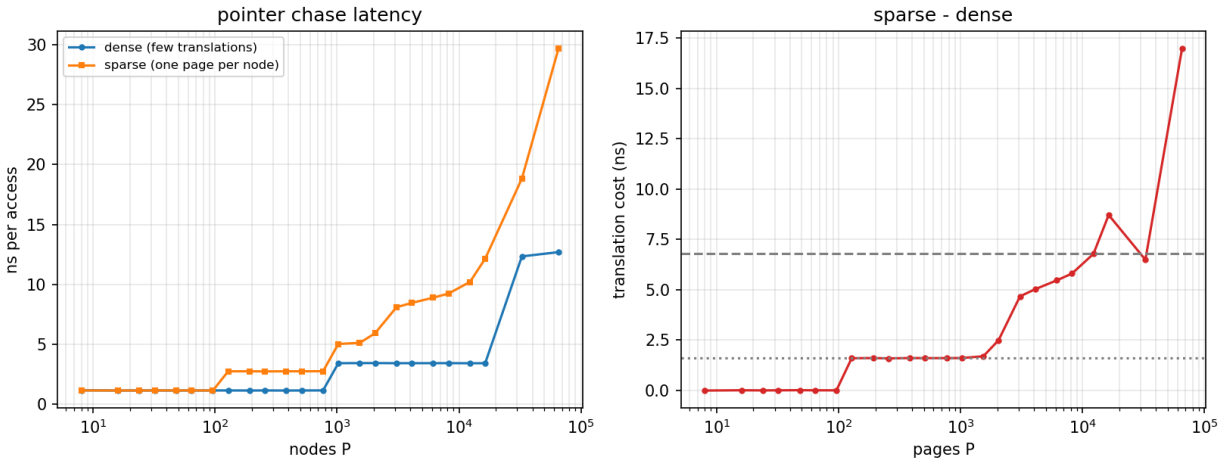


Figure 39: Elevation P-core TLB Miss Cost Data

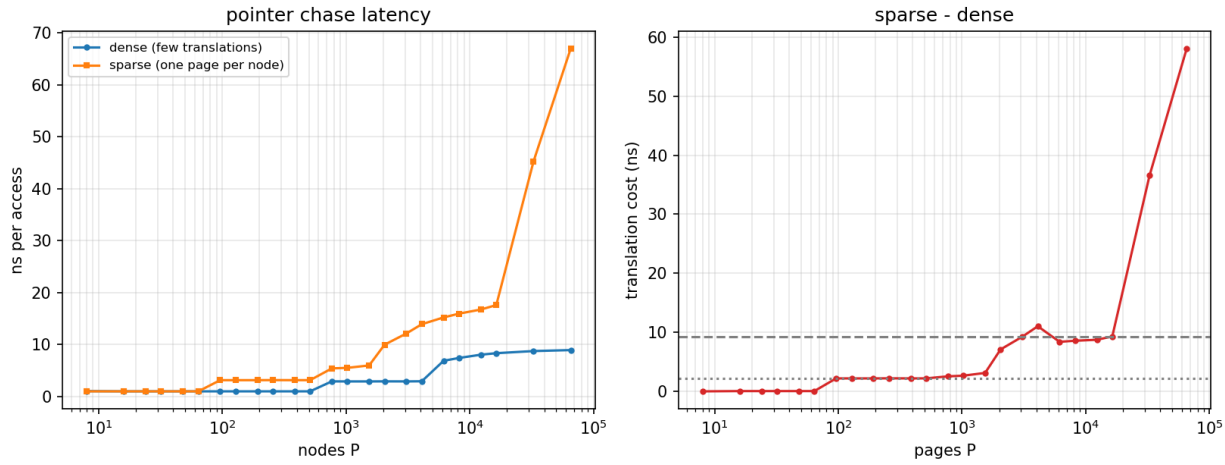


Figure 40: Ookay TLB Miss Cost Data

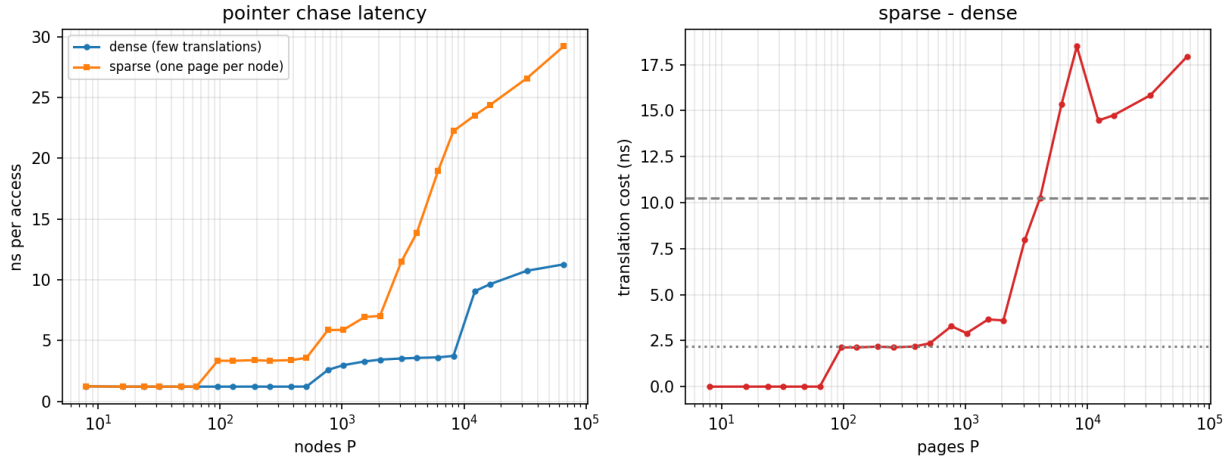


Figure 41: Skylark TLB Miss Cost Data

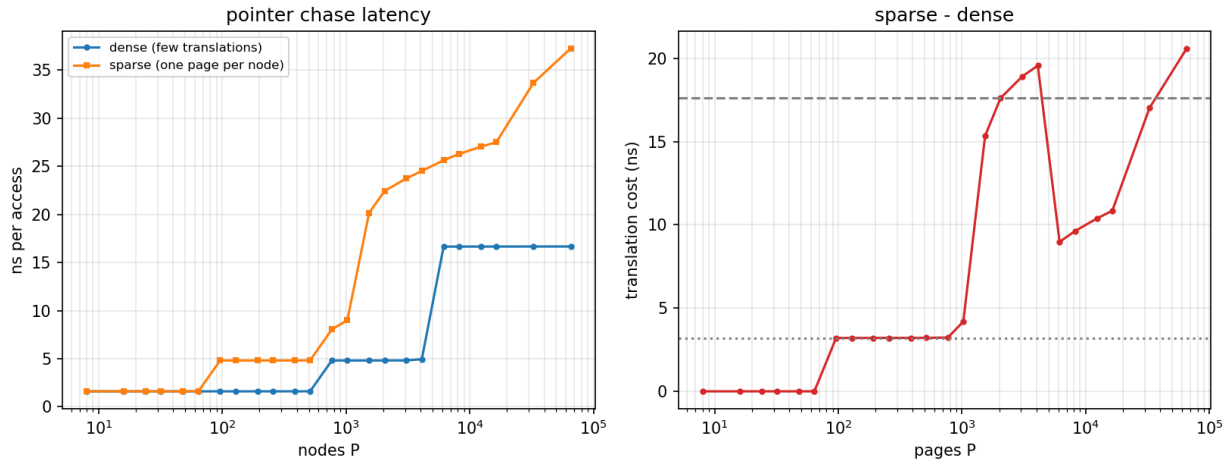


Figure 42: Sunbird TLB Miss Cost Execution Data

Analyzing the data, there is little difference in latency values between all the machines. However, the miss cycles in the SGX machine and p-cores of the hybrid machines are less than the other processors. This same trend applies to TLB Walk Cold Cycles and TLB Walk Cycles. The data for the normal walk cycles was more compacted than the data for the cold cycles, most likely due to the constant fetching the non-SGX processors had to do. A claim can be made that SGX-enabled and primary core hybrid processors are more efficient in TLB walking and have a lower miss penalty than non-SGX and non-hybrid processors.

Below, Listing 7, is the main code snippet used to test the TLB miss cost of each machine. This can be found in the repository at `CSC591-Project3‣exercises‣ex08_tlb_miss_cost‣tlb_test.c`

Listing 7: TLB Miss Cost Code Block

```
// tlb_test.c - What does a TLB miss cost? (x86-64, Linux)
//
// IDEA. Pointer-chase through P nodes in random order (every access depends on the previous one, so latency is exposed
// and prefetchers cannot help). Two layouts with the SAME number of nodes and the same cache footprint (P cache lines):
//   dense  : nodes packed 64 B apart (P*64 bytes; huge pages requested, so almost no TLB entries are needed)
//   sparse : one node per 4 KiB page (P pages -> P different translations); the node's offset inside the page varies
//            so the P lines spread evenly over the cache sets, like the dense layout
// Per-access time(sparse) - time(dense) isolates the cost of address translation as P grows:
//   P up to the L1 DTLB size : ~0     P beyond it: L1 DTLB miss, L2 TLB (STLB) hit    P beyond the STLB: page walk
// Interpretation caveat: for large P the page-table entries themselves may miss the caches, which is part of the cost.
//
// OUTPUT (CSV on stdout): P,layout,rep,ns_per_access

#include "bench_common.h"
#include <getopt.h>
#include <stdlib.h>
#include <string.h>
#include <sys/mman.h>

#if !defined(__x86_64__)
#error "tlb_test.c␣is␣x86-64␣only"
#endif

#define PAGE 4096UL
#define HUGE_SZ (2UL << 20)
#define MAXP 65536
static const int grid[] = {8, 16, 24, 32, 48, 64, 96, 128, 192, 256, 384, 512, 768, 1024, 1536, 2048,
                           3072, 4096, 6144, 8192, 12288, 16384, 32768, 65536};
#define NG (int)(sizeof(grid) / sizeof(grid[0]))

static uint64_t rng = 88172645463325252ULL;

// n dependent loads (n multiple of 8); returns the final pointer so the chain cannot be optimized away
__attribute__((noinline)) static void *chase(void *p, uint64_t n) {
    __asm__ volatile("1:\n\t"
                     ".rept␣8\n\t␣mov␣(%0),␣%0\n\t␣.endr\n\t"
                     "sub␣$8,␣%1\n\t"
                     "jnz␣1b\n\t"
                     : "+r"(p), "+r"(n)
                     :
```

```
                    : "cc", "memory");
    return p;
}

static void *map_region(size_t bytes, int huge) {
    char *raw = mmap(NULL, bytes + HUGE_SZ, PROT_READ | PROT_WRITE, MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
    if (raw == MAP_FAILED) { perror("mmap"); exit(1); }
    char *p = (char *)(((uintptr_t)raw + HUGE_SZ - 1) & ~(HUGE_SZ - 1));
    madvise(p, bytes, huge ? MADV_HUGEPAGE : MADV_NOHUGEPAGE);
    for (size_t i = 0; i < bytes; i += 4096) ((volatile char *)p)[i] = 0;    // fault everything in now
    return p;
}

static long anon_huge_kb(void) {           // how much of this process is really backed by huge pages
    FILE *f = fopen("/proc/self/smaps_rollup", "r");
    char line[256]; long kb = -1;
    if (!f) return -1;
    while (fgets(line, sizeof line, f)) if (sscanf(line, "AnonHugePages:␣%ld␣kB", &kb) == 1) break;
    fclose(f);
    return kb;
}

static void thp_setting(char *out, size_t n) {
    FILE *f = fopen("/sys/kernel/mm/transparent_hugepage/enabled", "r");
    char line[128] = "unknown";
    if (f) { if (!fgets(line, sizeof line, f)) strcpy(line, "unknown"); fclose(f); }
    char *l = strchr(line, '['), *r = l ? strchr(l, ']') : NULL;
    if (l && r) { *r = 0; snprintf(out, n, "%s", l + 1); } else snprintf(out, n, "unknown");
}

int main(int argc, char **argv) {
    int core = -1, reps = 9, c;
    uint64_t steps = 2000000;
    while ((c = getopt(argc, argv, "c:r:s:h")) != -1) {
        if (c == 'c') core = atoi(optarg);
        else if (c == 'r') reps = atoi(optarg);
        else if (c == 's') steps = strtoull(optarg, 0, 10) & ~7UL;
        else { fprintf(stderr, "usage:␣%s␣[-c␣core]␣[-r␣reps]␣[-s␣accesses_per_run]\n", argv[0]); return 1; }
    }
    if (core >= 0 && pin_to_cpu(core)) { perror("sched_setaffinity"); return 1; }

    char *sparse = map_region((size_t)MAXP * PAGE, 0);        // 4 KiB pages only
    char *dense = map_region((size_t)MAXP * 64, 1);           // huge pages if the kernel allows
    uint32_t *order = malloc(MAXP * sizeof(uint32_t));
    char thp[64]; thp_setting(thp, sizeof thp);

    printf("#␣arch=%s␣core=%d␣reps=%d␣steps=%lu␣thp=%s␣anon_huge_kb=%ld␣core_ghz=%.4f\n", BENCH_ARCH, core, reps,
           (unsigned long)steps, thp, anon_huge_kb(), core_ghz());
    printf("P,layout,rep,ns_per_access\n");
    for (int r = -1; r < reps; r++) {                          // rep-major, r=-1 warm-up
        for (int g = 0; g < NG; g++) {
            int P = grid[g];
            for (int i = 0; i < P; i++) order[i] = i;
            for (int i = P - 1; i > 0; i--) { int j = xorshift64(&rng) % (i + 1); uint32_t t = order[i]; order[i] = order[j]; order[j] = t; }
            for (int lay = 0; lay < 2; lay++) {               // 0 = dense, 1 = sparse
                char *base = lay ? sparse : dense;
                #define NODE(i) ((void **)(lay ? base + (size_t)(i) * PAGE + ((size_t)(i) % 64) * 64 : base + (size_t)(i) * 64))
                for (int k = 0; k < P; k++) *NODE(order[k]) = NODE(order[(k + 1) % P]);   // random cyclic chain
                void *p = NODE(order[0]);
                p = chase(p, (uint64_t)((P + 7) & ~7));       // one warm lap
                double t0 = now_ns();
                p = chase(p, steps);
                double t1 = now_ns();
                __asm__ volatile("" ::"r"(p));
                if (r >= 0) printf("%d,%s,%d,%.3f\n", P, lay ? "sparse" : "dense", r, (t1 - t0) / (double)steps);
            }
        }
        fflush(stdout);
    }
    return 0;
}
```

10 BTB Associativity

The last experiment conducted BTB associativity and capacity among the processors.. Each experiment was ran three times. At the end of the three test runs, data was collected and summarized into a table. The CSV summary table, Table 8, can be found in the repository at `CSC591-Project3 ▸ report ▸ tables ▸ ex10_btb_capacity_summary`. The figures shown below of the each of the machine's data is representation of the last test ran to analyze the BTB capacity. Each of the machines respective figures can be found in the repository at `CSC591-Project3 ▸ data ▸ <machine_name> ▸ ex10_btb_capacity ▸ <run_number> ▸ plot.png`

Machine	Core Label	BTB Capacity Est.	BTB Agree Across Strides	Base Cycles per Branch
artemisia	cpu29	16384	yes	1.85
charnwood	cpu2	4096	yes	1.6
crux	cpu3	4096	yes	1.2
descent	ecore	4096	no	0.95
descent	pcore	16384	yes	0.33
elevation	ecore	4096	no	1.0
elevation	pcore	16384	yes	0.34
ookay	cpu2	4096	yes	1.35
skylark	cpu2	4096	yes	1.0
sunbird	cpu8	4096	no	0.99

Table 9: Processor BTB Associativity Summary

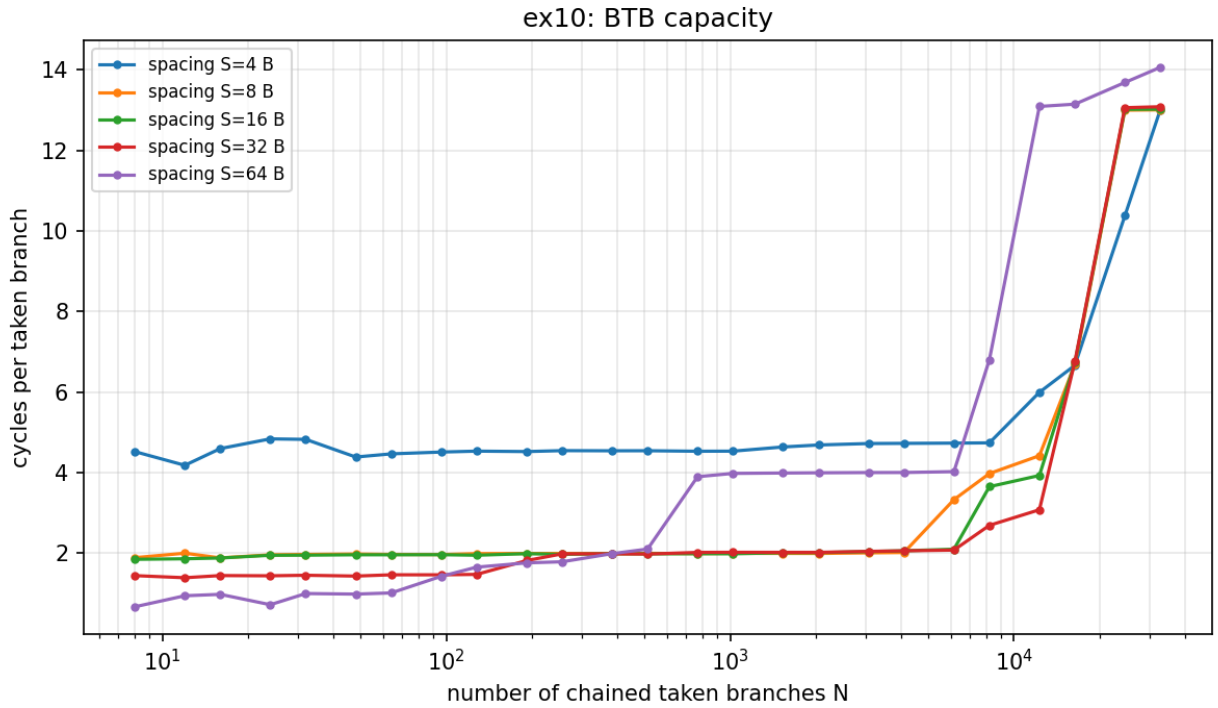


Figure 43: Artemisia BTB Associativity Data

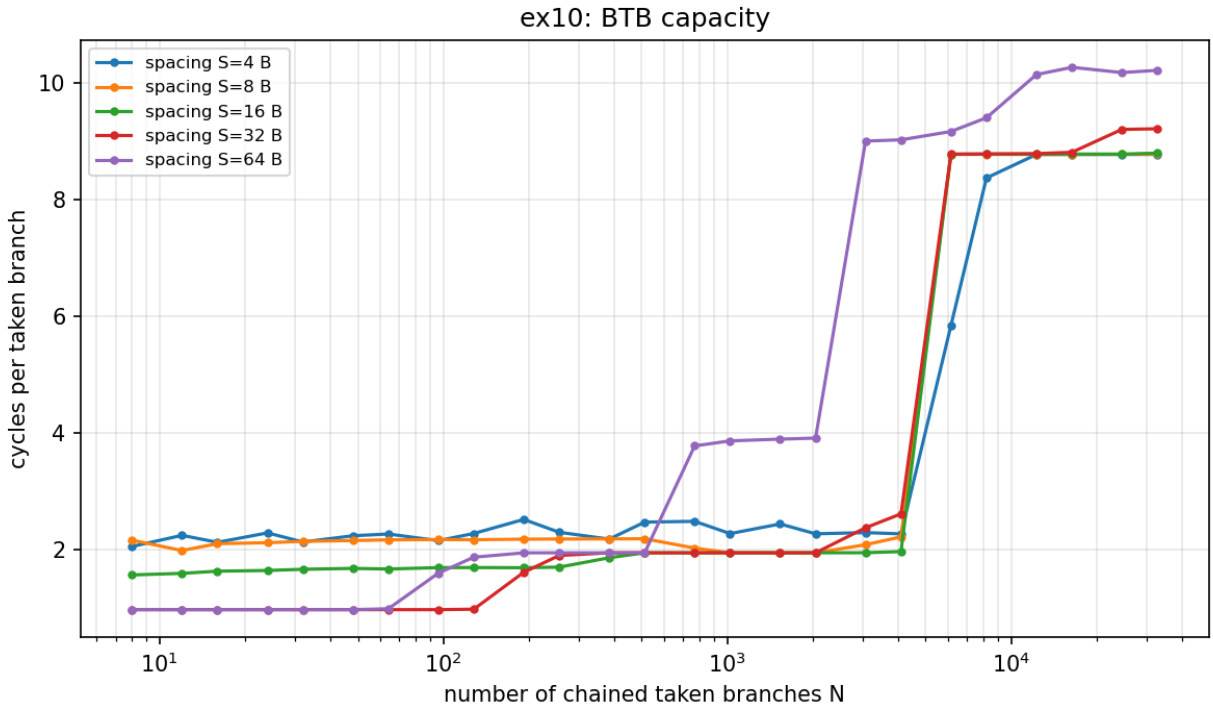


Figure 44: Charnwood BTB Associativity Data

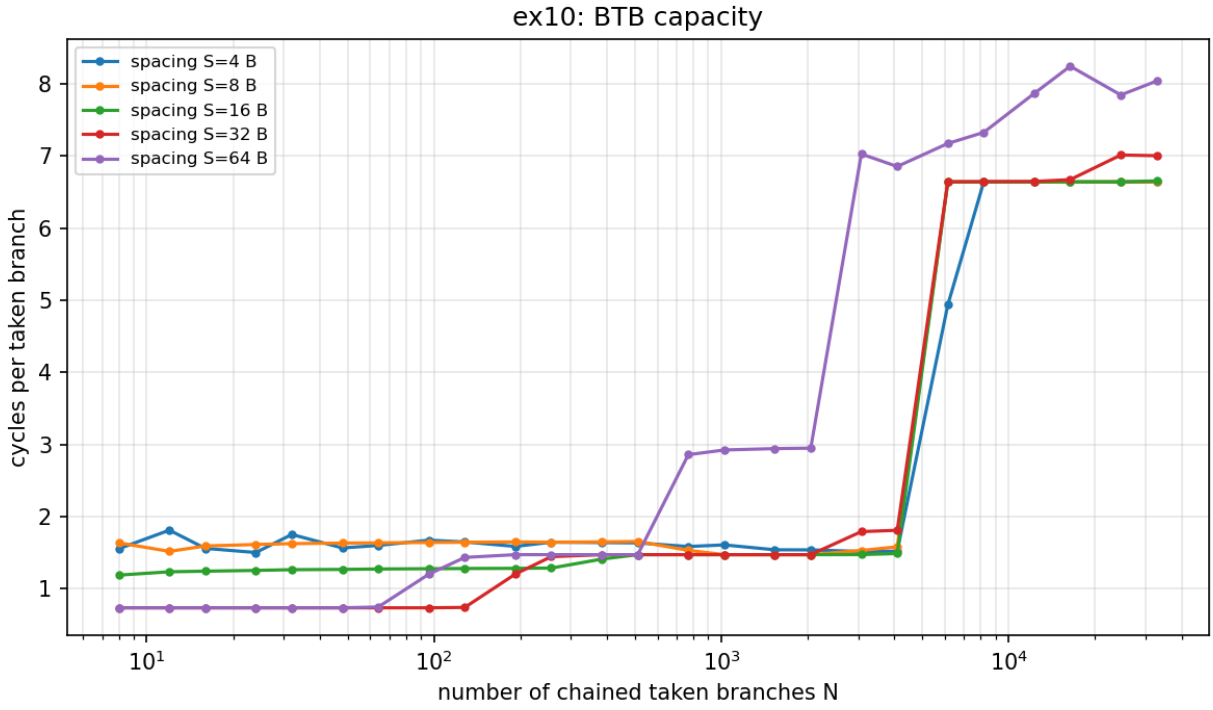


Figure 45: Crux BTB Associativity Data

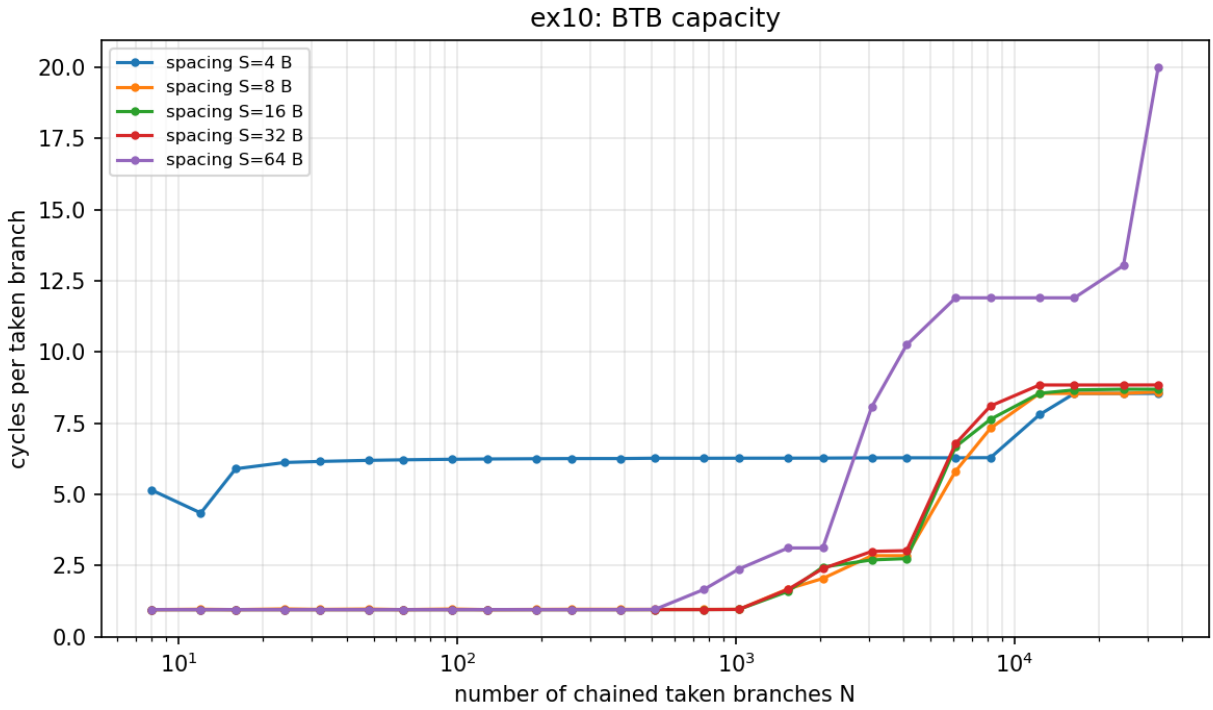


Figure 46: Descent E-core BTB Associativity Data

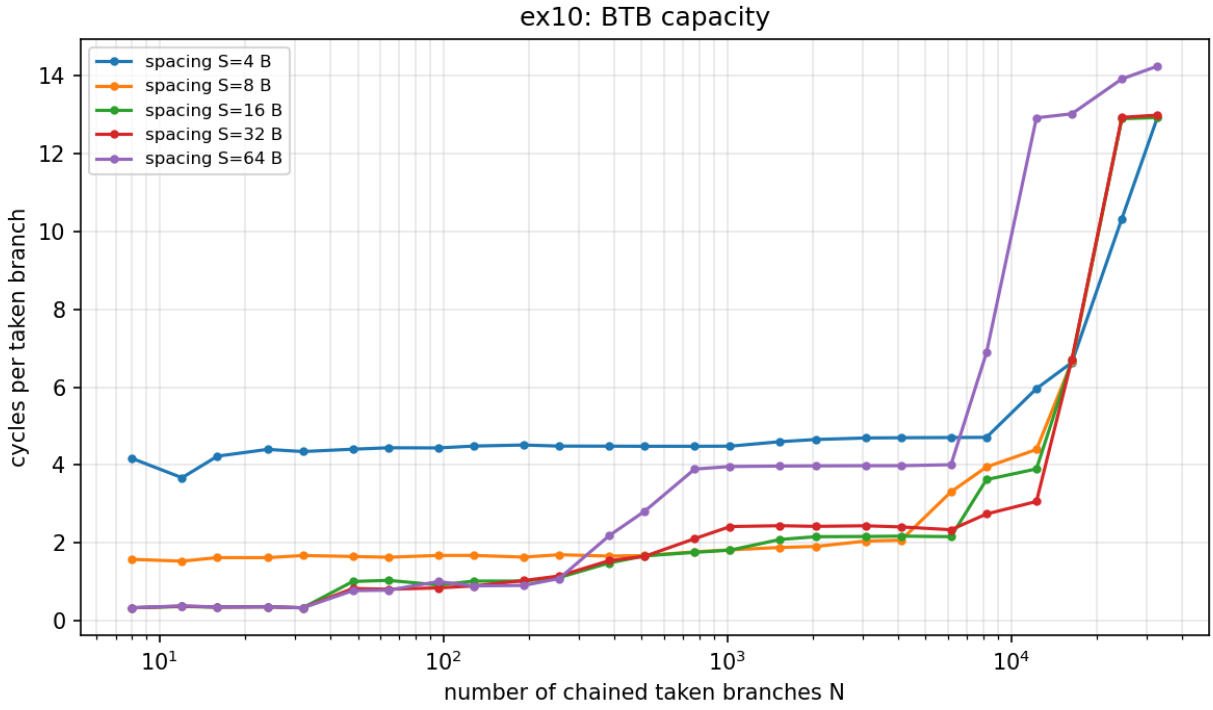


Figure 47: Descent P-core BTB Associativity Data

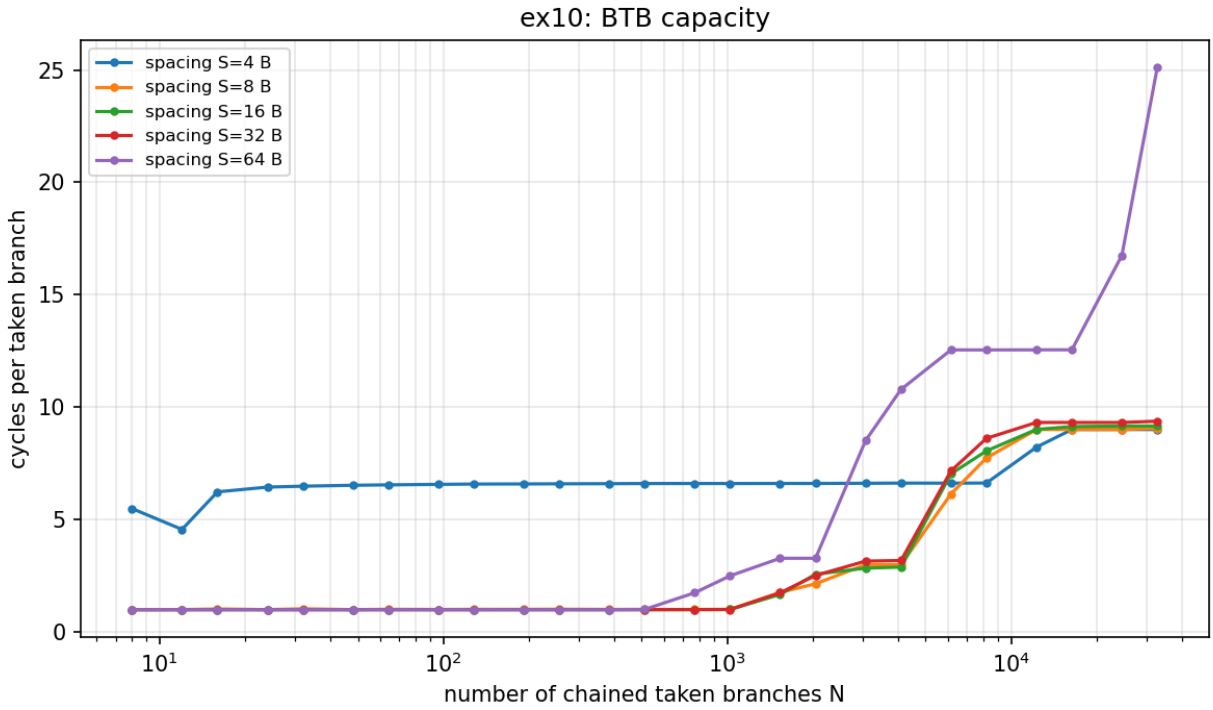


Figure 48: Elevation E-core BTB Associativity Data

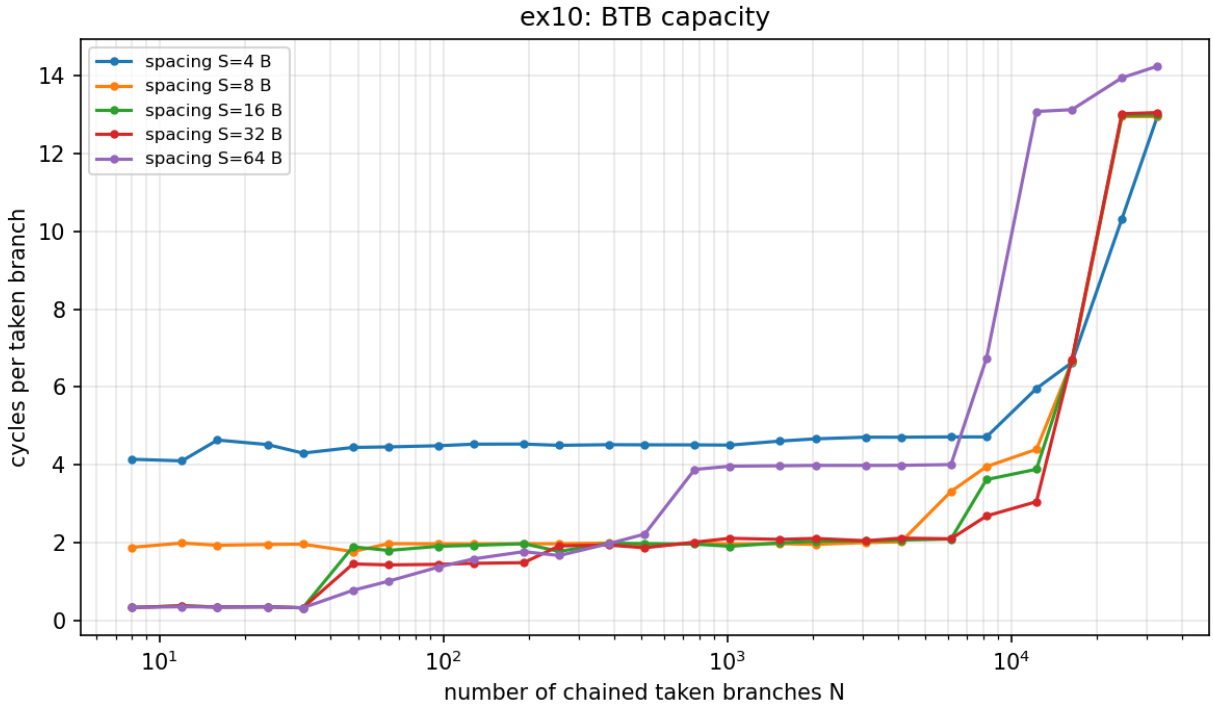


Figure 49: Elevation P-core BTB Associativity Data

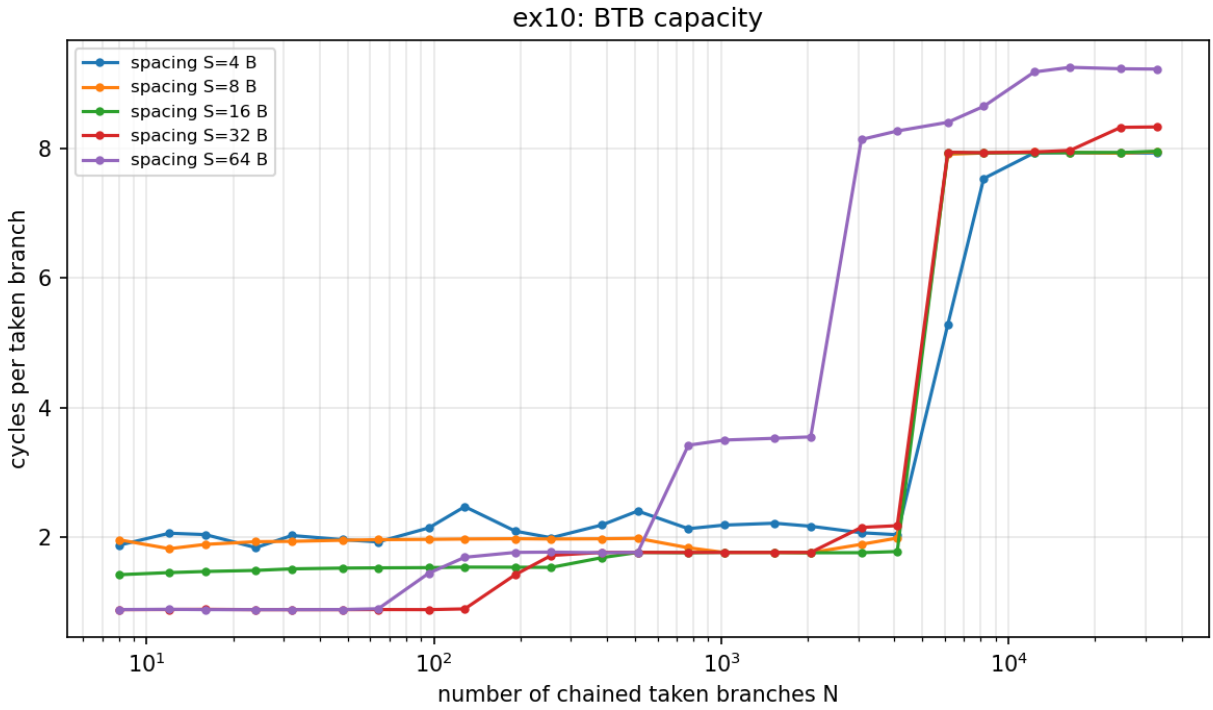


Figure 50: Ookay BTB Associativity Data

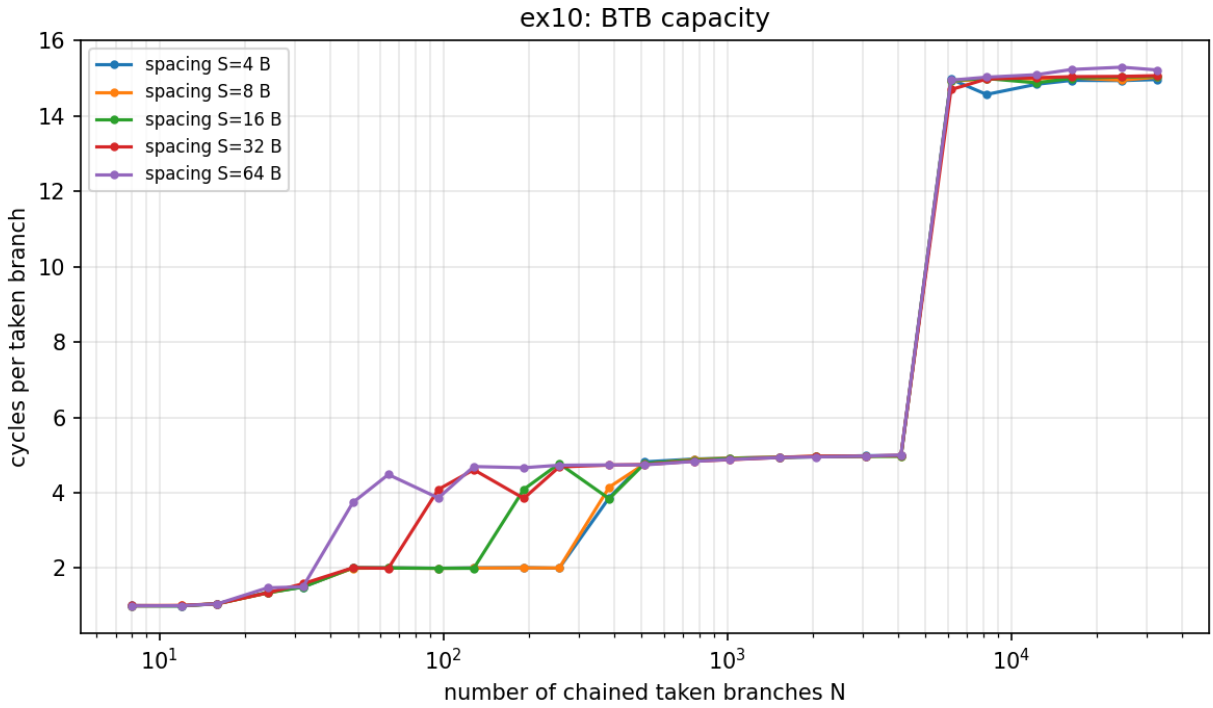


Figure 51: Skylark BTB Associativity Data

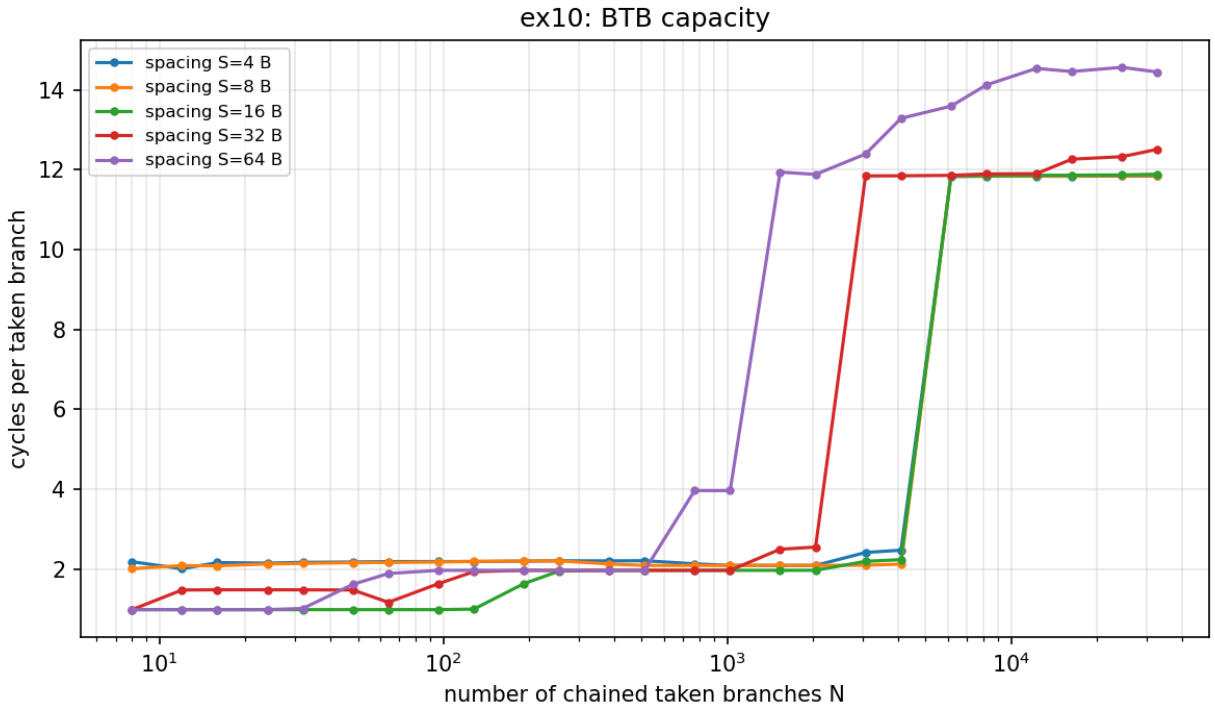


Figure 52: Sunbird BTB Associativity Execution Data

Analyzing the data, the graphs show the e-cores of the hybrid machines took far more cycles as the number of chain branches surpassed 10^4 than the other machines when the stride was 64 bytes. However, on the other strides, when the number of branches was larger, the number of cycles was condensed compare to other machines, even their own p-cores. Looking at the table, the SGX-enabled and hybrid processor's p-core have four times the size of BTB capacity than the other machines. Despite this larger capacity, the p-cores have the lowest base cycles per branch. This could be due to the size of the p-cores and the fetching time taking longer than other machines. Artemisia has the largest amount of base cycles per branch. A claim can be made, from the data, that SGX-enabled processors have a larger capacity and can execute more cycles per branch compared to non-SGX processors.

Below, Listing 8, is the main code snippet used to test the BTB associativity of each machine. This can be found in the repository at `CSC591-Project3‣exercises‣ex10_btb_capacity‣btb_test.c`

Listing 8: BTB Associativity Code Block

```
// btb_test.c - How many branches can the BTB (branch target buffer) hold? (x86-64, Linux)
//
// IDEA. Generate a chain of N unconditional jumps; each one jumps to the next, the last one returns to the first
// (through a counted loop). Every executed jump is a TAKEN branch whose target the front end must know early:
//   * if all N branches fit in the BTB, the front end redirects immediately: ~1 taken branch per cycle or two
//   * once N exceeds a BTB level, targets are not found early and each jump pays a re-steer bubble: time per branch steps up
// Branches are spaced S bytes apart (S = 4, 8, 16, 32, 64). The spacing changes how the branches map onto BTB sets and
// how much code the loop touches, so we measure several S: a step that moves with S points at cache/ITLB capacity,
// a step at the same N for every S points at a BTB capacity. The code is generated at run time (mmap + mprotect).
//   S=4 : EB 02 90 90                    (jmp +2 over two padding bytes -> next block)
//   S>=8: E9 rel32 (rel32 = S-5) + padding  (jmp to the next block)
//   end : dec %rdi ; jnz first_block ; ret  (the loop-closing branch is a taken branch too: N+1 taken branches/iteration)
//
// OUTPUT (CSV on stdout): S,N,rep,ns_per_branch

#include "bench_common.h"
#include <getopt.h>
#include <stdlib.h>
#include <string.h>
```

```c
#include <sys/mman.h>

#if !defined(__x86_64__)
#error "btb_test.c is x86-64 only"
#endif

static const int spacings[] = {4, 8, 16, 32, 64};
static const int grid[] = {8, 12, 16, 24, 32, 48, 64, 96, 128, 192, 256, 384, 512, 768, 1024, 1536, 2048, 3072,
                           4096, 6144, 8192, 12288, 16384, 24576, 32768};
#define NS (int)(sizeof(spacings) / sizeof(spacings[0]))
#define NG (int)(sizeof(grid) / sizeof(grid[0]))
typedef void (*fn_t)(uint64_t);

static fn_t build_chain(size_t n, size_t S) {
    size_t len = n * S + 16, maplen = (len + 4095) & ~4095UL;
    uint8_t *code = mmap(NULL, maplen, PROT_READ | PROT_WRITE, MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
    if (code == MAP_FAILED) { perror("mmap"); exit(1); }
    memset(code, 0x90, maplen);
    for (size_t i = 0; i < n; i++) {
        uint8_t *p = code + i * S;
        if (S == 4) { p[0] = 0xEB; p[1] = 0x02; }                        // jmp +2: skips the two padding bytes
        else { p[0] = 0xE9; int32_t rel = (int32_t)(S - 5); memcpy(p + 1, &rel, 4); }   // jmp to p + S
    }
    uint8_t *ep = code + n * S;                                           // loop end: dec rdi; jnz code; ret
    ep[0] = 0x48; ep[1] = 0xFF; ep[2] = 0xCF;
    ep[3] = 0x0F; ep[4] = 0x85;
    int32_t rel = (int32_t)(code - (ep + 9));
    memcpy(ep + 5, &rel, 4);
    ep[9] = 0xC3;
    if (mprotect(code, maplen, PROT_READ | PROT_EXEC)) {
        perror("mprotect(PROT_EXEC) (is this directory/system W^X-restricted?)"); exit(1);
    }
    return (fn_t)code;
}

int main(int argc, char **argv) {
    int core = -1, reps = 7, c;
    uint64_t target = 20000000;                    // taken branches per timed run
    while ((c = getopt(argc, argv, "c:r:t:h")) != -1) {
        if (c == 'c') core = atoi(optarg);
        else if (c == 'r') reps = atoi(optarg);
        else if (c == 't') target = strtoull(optarg, 0, 10);
        else { fprintf(stderr, "usage: %s [-c core] [-r reps] [-t taken_branches_per_run]\n", argv[0]); return 1; }
    }
    if (core >= 0 && pin_to_cpu(core)) { perror("sched_setaffinity"); return 1; }

    fn_t fn[NS][NG];
    for (int s = 0; s < NS; s++) for (int g = 0; g < NG; g++) fn[s][g] = build_chain(grid[g], spacings[s]);

    printf("# arch=%s core=%d reps=%d taken_per_run=%lu core_ghz=%.4f\n", BENCH_ARCH, core, reps, (unsigned long)target, core_ghz());
    printf("S,N,rep,ns_per_branch\n");
    for (int r = -1; r < reps; r++) {              // rep-major, r=-1 warm-up
        for (int s = 0; s < NS; s++) {
            for (int g = 0; g < NG; g++) {
                uint64_t per = (uint64_t)grid[g] + 1, iters = target / per;
                if (iters < 2) iters = 2;
                fn[s][g](2);                       // prime the front end / BTB with this exact chain
                double t0 = now_ns();
                fn[s][g](iters);
                double t1 = now_ns();
                if (r >= 0) printf("%d,%d,%d,%.4f\n", spacings[s], grid[g], r, (t1 - t0) / (double)(iters * per));
            }
        }
        fflush(stdout);
    }
    return 0;
}
```

11 AI Disclosure

Claude was used to assist in code creation for each exercise, as well as data analysis in the forms of charts and tables. All code was reviewed by the author for mistakes and personalized to fit their own experiments and machines. All background information, written data interpretation, and claims written in the report was written by the author with no AI assistance.